



Research Repository UCD

Title	A new implementation of the elliptic curve method of integer factorization using Edwards and Hessian curves
Authors(s)	Robinson, Oisin Matthew
Publication date	2015
Publication information	Robinson, Oisin Matthew. "A New Implementation of the Elliptic Curve Method of Integer Factorization Using Edwards and Hessian Curves." University College Dublin. School of Mathematics and Statistics , 2015.
Publisher	University College Dublin. School of Mathematics and Statistics
Item record/more information	http://hdl.handle.net/10197/8540

Downloaded 2026-05-14 19:52:32

The UCD community has made this article openly available. Please share how this access benefits you. Your story matters! (@ucd_oa)



© Some rights reserved. For more information

**A New Implementation of
the Elliptic Curve Method
of Integer Factorization
using Edwards and Hessian Curves**

Oisín Robinson

Student Number: 94045801

This thesis is submitted to University College Dublin in fulfilment of the requirements for the
degree of Research Masters in Mathematical Science

UCD School of Mathematical Science

Head of School: Professor Gary McGuire
Principal Supervisor: Professor Gary McGuire

August 2015

Contents

1	Introduction	1
1.1	Brief History of Factoring Methods	1
1.2	Improvements to ECM	4
1.3	Overview of this Thesis	4
2	ECM in Detail	5
2.1	Stage 1	5
2.1.1	Overview	5
2.1.2	The Stage 1 Scalar Multiple	8
2.1.3	Arithmetic of Large Integers - Theory and Practice	8
2.1.4	Montgomery Multiplication	10
2.1.5	Windowing	12
2.1.6	Curves with Small Parameters	12
2.1.7	An Obstruction	13
2.2	Stage 2	14
2.2.1	Naive Method	14
2.2.2	Birthday Paradox Continuation	15
2.2.3	FFT Continuation	16
2.2.4	Kronecker-Schönhage Polynomial Multiplication	19
2.2.5	Polynomial Product Tree	20
2.2.6	Bernstein Scaled Remainder Tree	21
2.2.7	Polynomial Pseudo-Inverse (Reciprocal)	24
2.2.8	Polynomial Modulo Reduction	25
2.2.9	Stage 2 Blocks	26
2.2.10	The Brent-Suyama Extension	28
3	Recent Improvements to ECM	30
3.1	Brief Overview of the Theory of Elliptic Curves	30
3.1.1	History and Definition	30
3.1.2	Algebraic Properties	31
3.1.3	The Mordell-Weil Theorem	31

3.1.4	The Lutz-Nagell Theorem	31
3.2	Torsion, Rank and Number Fields	32
3.2.1	Mazur's Theorem	32
3.2.2	The Search for a Generalization	33
3.2.3	Relevance to ECM	33
3.3	Edwards Curves	35
3.4	Hessian Curves	37
3.5	A Note on the Function ψ	38
3.6	The Jeon-Kim-Lee Families	40
3.6.1	Torsion Family $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$	40
3.6.2	Torsion Family $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$	41
3.6.3	A Coincidence	42
3.6.4	Visualization of Curves with Positive Rank	43
3.6.5	Other Torsion Families	45
3.7	Alternative - The Atkin-Morain Family	45
3.8	Hessian Curve Advantage in Scalar Multiplication	46
3.9	Even Larger Torsion	48
4	The Implementation	51
4.1	Overview	51
4.2	SAGE and Generating Curves with Positive Rank	52
4.3	A Problem Arising from Isogenies between Curves	52
4.4	The Development Process	54
4.4.1	Working with Pari/GP, C++, Java	54
4.4.2	Memory Leaks and Valgrind	54
4.4.3	A Note on Assembly Language	54
4.5	A Smartphone App	55
5	Computational Aspects	56
5.1	Which Numbers?	56
5.1.1	Cunningham Numbers	56
5.1.2	Other Numbers	57
5.2	Current Records and Record Attempts	57

5.3	Testing the Implementation on Fionn	58
5.4	Results	58
5.4.1	Speed	60
5.4.2	Effectiveness	62
5.4.3	Memory Usage	62
6	Conclusion	68
6.1	Generality of the Torsion Improvement	68
6.2	Further Possible Gains	69
6.3	Application in Various Scenarios	70
6.4	Questions that Remain Open	70
A	Appendix - Source Code, SAGE and Pari/GP Scripts etc	71
A.1	Sample C++ Function to Compute Hessian Scalar Multiple of Point, using GNU-MP Library	71
A.2	SAGE script to generate curves from $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family	73
A.3	SAGE script to generate curves from $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family	76
A.4	Sample Twisted Edwards Curves from $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family	78
A.5	Sample Twisted Hessian Curves from $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family	79
A.6	PARI/GP script to factor using stage 1 of ECM and $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves .	80
A.7	PARI/GP script to factor using stage 1 of ECM and $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curves .	81
	Bibliography	84

Abstract

In this thesis, three main ideas characterise a new implementation of the Elliptic Curve Method (ECM) of integer factorization. The first idea is the use of Edwards/Hessian curves for all elliptic curve computations, including stage 2. The second idea is the use of families of elliptic curves with larger torsion subgroup over quartic number fields, in particular $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ for a family of Edwards curves, and $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ for a family of Hessian curves. The third idea is the generation of respectively hundreds/a few thousand of such curves from families given by Jeon/Kim/Lee, which have small parameters and a point of infinite order having small projective coordinates, leading to improved efficiency in scalar multiplication. The curves are generated using SAGE. The FFT continuation for stage 2 is implemented. The performance of the software is analysed and compared to the leading implementations, in terms of effectiveness/speed/memory usage. The implementation is tested on ICHEC's Fionn cluster. The use of Hessian curves in an implementation of ECM appears new. A new discovery is that EECM-MPFQ's 'good curves' for ECM having torsion $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ are a subset of the Jeon-Kim-Lee $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family, yielding many hundreds more good curves than the 100 from two torsion families produced for EECM-MPFQ, not to mention several thousand curves from the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family with small parameters. This remedies one drawback of EECM-MPFQ - lack of good curves. Another discovery is that, for small-parameter Hessian curves, the speed of the addition formula is particularly fast, allowing Hessian scalar multiplication without windowing to compete with Edwards scalar multiplication with windowing. Since windowing is not required, the associated higher memory cost is not incurred. This has a beneficial consequence for ECM in memory-constrained environments such as when implemented on GPUs. This in turn may benefit the sieving stage of the number field sieve.

I hereby certify that the submitted work is my own work, was completed while registered as a candidate for the degree stated on the Title Page, and I have not obtained a degree elsewhere on the basis of the research presented in this submitted work.

Acknowledgements

The author would like to thank Professor Gary McGuire for support and direction given.

1 Introduction

1.1 Brief History of Factoring Methods

The integer factorisation problem is simply to determine the unique prime factorisation of an arbitrary integer as efficiently as possible. Most progress on the integer factorisation problem has been made within the last half-century, which has seen the development of all known methods with (conjectured or rigorously proved) subexponential, as opposed to exponential, complexity. In general, no polynomial time algorithm is known at present. Prior to this, large numbers could be factored, but only in certain cases - they were conditional factoring methods. An example is Fermat's factoring method, which works only if two factors of the target number are not too far apart. Another example is Euler's factoring method, which only works as long as two representations of a number as a sum of two squares are known.

However, in the 20th century, new ideas began to emerge (some of which may have been hinted at in the 19th century) that have led to much faster algorithms than anything previously generally known. Kraitchik, in the 1920s, for example, suggested looking not for a difference of squares, as in Fermat's method, but for a congruence of squares

$$x^2 \equiv y^2 \pmod{n}$$

since then

$$x^2 - y^2 = (x - y)(x + y) = kn$$

for some k . We can then expect $x - y$ to have about a 50% chance of sharing a non-trivial (not 1 or n) factor with n . One advantage of this is that any numbers x and y that we now need to consider are bounded by n . Another advantage is that it facilitates the modern insight of the use of so-called 'smooth numbers' that may be related to the number being factored. This approach was pioneered by Morrison and Brillhart in the 1970s. We start out with only one side of the congruence of squares, and look for many 'relations'

$$x^2 \equiv s_i \pmod{n} \qquad i = 1, \dots, k + 1$$

where the numbers s_i are made up of only small primes (and prime powers), in a 'factor base' of k primes. We should only choose primes which are squares mod n , since otherwise the method would become awkward/unworkable. The fact that we have $k + 1$ relations allows us to use a fact from linear algebra:

Theorem 1. *There exists a linear dependency among any $k + 1$ nonzero vectors of a k -dimensional vector space.*

Proof. See any introductory course on linear algebra.

□

We already have a square on the left of the congruence. We are trying to construct a

square on the right. Whereas each relation is unlikely to give a square on the rhs (the rhs just looks like a product of random small prime powers), if we multiply a certain selection of relations together, on the rhs and lhs, the lhs will remain a square, but it might happen that all the powers on the rhs sum to only even numbers - if this happens we have a square, since any even number is divisible by two. So we consider the powers on the rhs modulo 2. This is where we get a vector space (recall that a vector space is defined over a field, so we need a field. The set of numbers modulo 2 is a field). So, the vector space is $(\mathbb{F}_2)^k$. What is more, the above fact means that if we have found $k + 1$ vectors, we are guaranteed to have a linear dependency - which means that some linear combination of vectors is the zero vector. Since the vectors correspond to powers of the k primes, this amounts to guaranteeing that the product of some selection of the relations results in an even total power on the rhs - giving a square on the right. The question of how to find relations in the first place is what distinguishes one modern factoring method (that uses this approach) from the other.

The continued fraction algorithm (CFRAC), due to Michael A. Morrison and John Brillhart, guarantees being able to find many numbers that are squares mod n , and are not greater than $2\sqrt{n}$ (and hence are more likely to be smooth - only divisible by small prime powers - than numbers of the size of n , since the proportion of smooth numbers decreases with n).

The quadratic sieve (QS), due to Carl Pomerance ([Pom85], [Pom08]), combines finding relations via arithmetic progressions of a quadratic polynomial with a particularly fast method of detecting smooth numbers - sieving.

The elliptic curve method (ECM), due to Hendrik Lenstra Jr [Len87], seeks the identity in an elliptic curve group $E(\mathbb{F}_p)$, for $p|n$ (using only information mod n), as opposed to seeking a congruence of squares. In fact under a plausible assumption in its complexity analysis, it is the best method known of finding factors of ‘unbalanced’ composites - numbers with prime factors of very unequal size.

With the benefit of hindsight it is possible for us to compare the operation of ECM to that of a previously published algorithm [Pol74] - John Pollard’s ‘P - 1’ algorithm. Both ECM and the P - 1 algorithm can be loosely described as computing a large power of an element in a group, producing a value that has a factor in common with n . In the case of the P - 1 method, the group is $\mathbb{F}_p$, where p is the prime factor being sought. In the case of ECM, the group is an elliptic curve over $\mathbb{F}_p$, where p is again a prime divisor of n . In both cases the actual group is a priori unknown - but in both cases it is possible to work in the ring $\mathbb{Z}/n\mathbb{Z}$ instead and effectively still compute the large group element power in all subfields of the ring at once, in the hope that we may differentiate between factors of n by the group operation behaving differently in some subfields than others. ECM is described in more detail in section 2.

Returning to the congruence of squares approach, a collaboration of mathematicians around the start of the 1990s, from an idea of Pollard, produced the general number field sieve (GNFS, [LL93]). Here, the numbers tested for smoothness have even fewer digits than the $(\log 2 + 1/2 \log n)$ upper bound of the continued fraction method - the number of digits in the GNFS case being of the order of $(\log(n))^{2/3}$ - hence an even greater likelihood of smoothness.

It is possible to examine factorisation methods in terms of their complexity. CFRAC, QS, ECM and GNFS, under assumptions of very plausible conjectures, all have ‘subexponential’ complexity - asymptotically, they are slower than any polynomial time algorithm, but faster than any exponential time algorithm.

We define ‘L-notation’ as a measure of subexponential complexity.

Definition 1. *Let n be a positive integer. Define*

$$L(t, c) = e^{(c + \mathcal{O}(1))(\log n)^t (\log \log n)^{1-t}}$$

As $n \rightarrow \infty$, an algorithm with complexity $L(1, c)$ is exponential for any c , with complexity $L(0, c)$ is polynomial for any c , and with complexity $L(t, c)$ for $0 < t < 1$ is subexponential for any c .

Let n be a positive integer we are trying to factor.

Conjecture 1. *The complexity of the CFRAC factoring algorithm is $L(1/2, \sqrt{2})$*

Conjecture 2. *The complexity of the QS factoring algorithm is $L(1/2, 1)$*

Conjecture 3. *The complexity of the ECM factoring algorithm, to find a factor p of n is*

$$\mathcal{O} \left(e^{(1 + \mathcal{O}(1)) \sqrt{(\log p \log \log p)}} \right)$$

Remark 1. We can see that the complexity of ECM only depends on p , and so is independent of the size of n . This gives ECM an advantage over other factoring methods mentioned. Numbers whose size puts them out of reach of the number field sieve (for example) are amenable to factorisation by ECM, provided the factors are small enough.

Conjecture 4. *The complexity of the GNFS factoring algorithm is $L(1/3, (64/9)^{1/3})$*

Under these conjectures, the general number field sieve is still asymptotically the fastest known method for factoring numbers not of a special form. A wider view of the integer factorisation problem is possible now. The best algorithms seem to end up with complexity $L(1/2)$ or $L(1/3)$. A general-purpose $L(1/4)$ algorithm, for example, would be a major theoretical breakthrough. The true complexity of integer factorisation is an open question.

1.2 Improvements to ECM

There have been a number of practical improvements made to ECM since its original publication. These include methods to both improve speed, and effectiveness. In the operation of ECM, different elliptic curves are selected until one curve computation yields a factor. Improving speed will reduce the time taken per curve. Improving effectiveness will reduce the number of curves that do not yield a factorisation. Note that the complexity of the algorithm itself has not been improved - hence any improvements are by a log factor of p at best. See the next section for more detail on the major improvements to ECM.

1.3 Overview of this Thesis

Since a primary motivation of this thesis was to create and test an implementation, aspects of this are described at length in section 2. However, this is background material and any new discoveries made are covered in later sections. Thus, section 2 would be of most use as a guide to implementation. Section 3 covers the relevant material relating to elliptic curves and describes the main discoveries of this work. Section 4 describes the implementation process, including a practical consideration regarding isogenies between curves that needed to be addressed. Section 5 deals with the results of testing the implementation at length in a high-end computing environment, with many thousands of hours in total (across multiple cpus) invested in factoring attempts using large numbers. The largest factors found are presented, and software timings are given and compared with the leading existing software. Finally section 6 concludes with a summary and lists some worthwhile future investigations/open questions. The appendices contain sample source code/scripts, and the parameters of some curves with good properties found (a small subset of the total number, for illustrative purposes).

2 ECM in Detail

2.1 Stage 1

2.1.1 Overview

ECM could be viewed as a way of transforming the factorization of a difficult-to-factor composite number N to the factorization of an easy-to-factor number $\#E(\mathbb{F}_p)$ - with one caveat - we do not know the number $\#E(\mathbb{F}_p)$ with the easier factorization. However, the great ability of Lenstra's algorithm to account for all possible $\#E(\mathbb{F}_p)$'s up to certain parameter bounds leads to being able to recover a non-trivial factor of N via the gcd algorithm.

Lenstra's original paper [Len87] detailed what is now referred to as 'stage 1' of ECM. We start by choosing an elliptic curve (see section 3.1.1 for a definition)

$$E : y^2 = x^3 + Ax + B$$

and find¹ a point (x, y) on E , where x and y are rational numbers, and A and B are usually integers (but always rational). For a reason to be explained below, we work with the curve in so-called "projective" (as opposed to the former "affine") form

$$E : ZY^2 = X^3 + AXZ^2 + BZ^3$$

together with the found point $(X : Y : Z)$ which satisfies the projective curve equation, where X , Y and Z are integers. If we know that (x, y) lies on the affine curve, we can compute $(X : Y : Z)$ by clearing the denominators of x and y , i.e. if

$$(x, y) = \left(\frac{a}{b}, \frac{c}{d}\right)$$

where

$$a, b, c, d \in \mathbb{Z}, \gcd(a, b) = \gcd(c, d) = 1, bd \neq 0$$

then

$$(X : Y : Z) = (ad : bc : bd)$$

where we divide all coordinates by $\gcd(ad, bc, bd)$ to minimize the integers that we have to work with (note that these coordinates are reused very frequently). Projective coordinates allow multiplying by any non-zero integer and the result is equivalent (although this is not the reason they are used - see below).

Then we make use of the fact that an elliptic curve has a group structure (and although the group operation is usually described as 'additive', it is not just simply the addition

¹It may be preferable to choose a point (x, y) first and deduce the curve parameters A and B , as explained in remark 3.

of two numbers. It is an operation which takes two points on the curve, and gives a third point on the curve. The coordinates of the third point are computed using formulas involving the X , Y and Z coordinates of the first two points). We ‘add’ the point $P = (X : Y : Z)$ to itself many times. If X , Y and Z are integers, there is no limit to how large the resulting coordinates can get. If P is chosen properly, the group operation over $\mathbb{Q}$ must not ever result in the group identity element $(0 : 1 : 0)$. This is the same as requiring that the point P ‘has infinite order in $E/\mathbb{Q}$ ’, which must be arranged in advance.

However, the situation changes to our advantage when we switch from considering E defined over $\mathbb{Q}$, to considering E defined over the ring $\mathbb{Z}/N\mathbb{Z}$ where N is the number to be factored. In this ring, all coordinate values lie in the range $0, \dots, N - 1$. Whereas before, the coordinates would quickly grow to sizes that would be difficult to work with, upon switching rings, the point is comparatively easy to work with. Switching rings also allows us to ensure we can obtain a point that satisfies the equation of $E/\mathbb{F}_p$, for any prime p (or indeed satisfies the equation of $E/\mathbb{Z}/N\mathbb{Z}$). All we have to do is reduce the coordinates of P , which are rational or projective integers, to values mod p (or mod N).

Remark 2. An elliptic curve defined over a finite field, written $E/\mathbb{F}_{p^e}$, has the structure of a group (See section 3.1). Note that an elliptic curve defined over a general ring no longer necessarily has a group structure, however groups are still at play because for each prime power p^e dividing the number N , $E/\mathbb{F}_{p^e}$ does have a group structure. Each $\mathbb{F}_{p^e}$ is a finite field.

The advantage comes with the fact that every point on an elliptic curve over a finite field has finite order. In other words, if we add P to itself k times, where k is the order of the point, we will get the identity element. But this is good news, because the identity element in $E/\mathbb{F}_p$ is $(0 : 1 : 0) \pmod{p}$, where p is a non-trivial factor of N . If we are any way close to having an expression for some $X \equiv 0 \pmod{p}$ it is a good situation - because we can recover p using $p = \gcd(X, N)$. In fact the quantity X can be taken as the X coordinate of a point on the elliptic curve that is equal to the identity in the group $E/\mathbb{F}_p$, but importantly, *not* equal to the identity in $E/\mathbb{F}_q$, where q is the cofactor of p dividing N (if $X \equiv 0 \pmod{q}$ as well, the gcd will just give back N).

The factors of N are not known in advance, so we don’t know which finite fields $\mathbb{F}_{p^e}$ are related to the structure $E/\mathbb{Z}/N\mathbb{Z}$. However, it turns out we can compute

$$[S]P = \underbrace{P + P + P + \dots + P}_{S \text{ times}}$$

and be sure the result will be the identity as long as S is a multiple of k , the order of P . How can this be arranged? We set S to be equal to a large number that is a multiple of many smaller numbers (for example the factorial function). Then many possible orders for P divide S . Although this will mean the number S will be large, there is a certain

range we can still make use of. The number of bits in S might even be in the millions, but even a computation of this size is not infeasible. There is a limit, it becomes increasingly difficult to compute $[S]P$ where $S = B!$, say, for some B which we hope is at least the size of the largest prime factor of k , as B grows even linearly at the higher end of the range. There is a useful range nonetheless. It may seem odd to say that computing this ‘scalar multiple’ of P is feasible, when even the number of *bits* in S is in the millions - but it is possible with the ‘double and add’ algorithm (see section 2.1.2), which goes through S bit-by-bit, so no longer are we computing via the magnitude of S , but via its representation.

Remark 3. Note that it is easier to switch the order given above, that is to say compute a projective point $(X : Y : Z)$ and deduce A and B than to do what is described above, i.e. choose curve first, then find a point satisfying the equation of the curve - unless we use what is usually referred to as a parametrised ‘family’ of curves where setting a parameter allows us to compute a point on the curve, and the curve coefficients (A and B), using formulas which depend only on the parameter. It is possible to find rational points on an elliptic curve having infinitely many rational points (which incidentally, conjecturally and asymptotically occurs 50% of the time), when they are within reach, by carrying out enhanced brute-force searches (in fact that is a central utilisation of this thesis), but this is not a deterministic process and cannot realistically be part of a fast-running algorithm at present - at best a large number of curves (and usually this is needed) must be precomputed, possibly requiring weeks of computer time.

Remark 4. However the curve E and point P are obtained, the point P must have infinite order in $E(\mathbb{Q})$ (the group of points on the curve over $\mathbb{Q}$). The group structure of an elliptic curve (see the Mordell-Weil Theorem, described in section 3.1.3) allows for points on $E/\mathbb{Q}$ of finite order, but over $\mathbb{Q}$, the maximum order, by Mazur’s ‘Torsion’ theorem (see 3.2.1) is 16. It would be wonderful to only have to compute at most $[16]P$ and be able to factor N , but alas, another theorem (see 3.2.3) means that, with one minor technicality, finite orders of a point will divide not only $\#E(\mathbb{F}_p)$ but $\#E(\mathbb{F}_q)$ as well - and obviously we have lost any distinction between p and q . It is for this reason that a point of infinite order over $\mathbb{Q}$ is required.

Remark 5. Using projective coordinates allows us to avoid having to do inversions mod N , which are usually otherwise an unavoidable consequence of applying a group law defined over $\mathbb{Q}$ to integers in a field $\mathbb{Z}/p\mathbb{Z}$. For example, the Edwards group law (see section 3.3) is reproduced here:

$$(x_1, y_1) \oplus (x_2, y_2) = \left(\frac{x_1 y_2 + x_2 y_1}{1 + dx_1 y_1 x_2 y_2}, \frac{y_1 y_2 - x_1 x_2}{1 - dx_1 y_1 x_2 y_2} \right)$$

where $(x_1, y_1), (x_2, y_2) \in \mathbb{Q} \times \mathbb{Q}$. This allows us to carry out Edwards point additions over $\mathbb{Z}/p\mathbb{Z}$ by using the same formula, except it is necessary to do modular inversions (an expensive operation compared to, say, modular multiplication). However, if the points we are adding are expressed in projective form we find that inversions are no longer neces-

sary - in fact the addition law can be expressed in ‘projective form’ as well by clearing denominators (this is usually a first step in producing the group law formulas as they are normally presented).

2.1.2 The Stage 1 Scalar Multiple

A key algorithm that must be implemented in stage 1 of ECM is the ‘double and add’ method of computing the large scalar multiple S of a fixed point P on an elliptic curve E . This proceeds by computing the binary representation of S , then starting with the point P we progress through the bits of S from LSB to MSB one-by-one, doubling each time, and also adding P if the bit is 1 (with no extra operation if the bit is 0). It is possible to compute the entire binary expansion of S all in one go, or alternatively proceed on a prime-by-prime basis - or indeed a batch of primes at a time. It saves some time ultimately spent on elliptic curve operations to compute S all in one go, but to compute S itself in reasonable time, the product tree algorithm for multiplication of large numbers of integers should be used (see section 2.2.5 for an analogous algorithm to multiply large numbers of polynomials together). This takes approximately $\mathcal{O}(k \log k)$ memory, where k is the number of primes in the product producing S , so memory usage becomes a consideration in computing very large S (the time required also then becomes non-negligible). Computing batches of primes may be a good compromise for very large stage 1 bounds.

We can expect the number of additions to be roughly half the number of doublings in this algorithm (a doubling is done for every bit of S , then for every set bit, of which comprise asymptotically 50% of bits, there is also an addition). The cost of additions is significant because typically an elliptic curve addition is more expensive than an elliptic curve point doubling. Windowing methods are an attempt to minimize the number of additions required, by changing the sequence of doublings and additions and introducing other operations, e.g. subtractions/triplings, to compute $[S]P$ without using the binary representation of S , but with the same net result (see section 2.1.5). However, in this thesis, in the case of twisted Hessian curves, it was discovered that it is possible to minimize the cost of additions in a different way - by making each addition itself less expensive by finding a point with small coordinates, which makes a certain twisted Hessian addition formula particularly efficient (see section 2.1.6).

2.1.3 Arithmetic of Large Integers - Theory and Practice

Factoring large integers (e.g. larger than 2^{64}) requires the storage and manipulation of large integers. Usually this is not built in to the common programming languages (including common C++ compilers), and a third-party library must be used. There are a number of libraries that provide the ability to manipulate large integers, the most well known probably being GMP - although there are others, such as MPFQ which was used in the EECM-MPFQ implementation [BBLP13] of ECM. While addition and subtraction of

large integers is straightforward in theory and in practice, other operations such as multiplication, division, reduction mod N , gcd, etc. are not necessarily so straightforward. The multiplication of large integers in particular is central in many computational scenarios, including ECM, and warrants further examination.

It is possible to multiply integers on a computer using the method for multiplication taught in school - the ‘schoolbook algorithm’. It has complexity $\mathcal{O}(N^2)$, where N is the (equal) number of digits of the two integers that we are multiplying. In fact this method is also a practical one - and it is indeed used to multiply integers up to a certain size on a computer, and further, is faster even than some more advanced methods for integers in its range of application (up to a couple of hundred digits). However, asymptotically, better algorithms exist. We can pose the question, “What is the complexity of integer multiplication?”. Whereas the absolute complexity is not currently known, various improvements to the known lower complexity bound have been made over the past few decades, with a precedent set by Schönhage-Strassen [SS71] in 1971, where the authors give an algorithm which multiplies integers using FFT techniques with asymptotic complexity $\mathcal{O}(N \log N \log \log N)$. There is a useful lower limit to the size of integers with which the algorithm is faster than the schoolbook algorithm. The lower bound was not bettered until 2009 [Für09], when Fürer obtained the asymptotic complexity $\mathcal{O}(N \log N 2^{\mathcal{O}(\log^* N)})$ for integer multiplication. Here, $\log^* N$ is the ‘iterated logarithm’ function, which is an integer equal to the number of times log must be applied until the result is not greater than unity. However, this advance, unlike Schönhage-Strassen’s algorithm, does not yield a practical method because of the constants in the complexity. Fürer’s algorithm only becomes more efficient at numbers so large that they would be of no use in ECM factoring (for example). Nonetheless, it is an active area of research to obtain further improvements, both theoretical and practical.

Remark 6. Schönhage-Strassen’s FFT multiplication makes a valuable contribution to stage 2 of ECM (see section 2.2.4). In fact, the library GMP was chosen for the implementation precisely because it carries out Schönhage-Strassen FFT multiplication automatically for very large integers.

Remark 7. Another frequently used operation is reduction mod N . Typically, in cryptographic and cryptanalytic applications, numbers are computed modulo a prime or composite number. For example, in Pollard’s $p - 1$ factoring algorithm, it is necessary to exponentiate an integer modulo the number N to be factored, to some large power. This would be infeasible (to even store) if normal integer exponentiation was used. But by successive squaring, and multiplying, each time reducing the answer mod N , the calculations always produce a number between 0 and $N - 1$. Modular reduction features heavily in the computation of elliptic curve group operations. There are formulas to calculate the coordinates of the sum of two points, and these formulas feature the addition, subtraction and multiplication of point coordinates, all of which must be reduced modulo a

prime or composite number. Each individual multiplication will typically be of two integers the size of N , resulting in an integer of size N^2 , which must then be reduced mod N .

2.1.4 Montgomery Multiplication

Optimising ECM involves identifying bottlenecks and other parts of the algorithm that are inefficient, and substituting faster techniques. One potential performance degradation is the computing of large numbers of modular reductions. We have seen how arithmetic mod N can involve an operation that results in a number larger than N , which must then be reduced to its mod N residue - a number between 0 and $N - 1$. A straightforward approach to this computation is to divide the result R of a computation by N (integer division), and set the mod N residue r to the remainder after the division. However, integer division is slower in practice than modular multiplication and we would like to avoid it if we can.

An improvement to this situation is to use an alternative representation for the numbers we work with - ‘Montgomery representation’. When integers are represented in this form, modular reductions are much more efficient - integer division is no longer necessary, and the operation that allows computing the mod N reductions is bit shifting (together with two multiplications and at most two additions). Since a division precomputation is needed, transforming to and from Montgomery representation, with a reduction, would make a single mod N reduction less efficient, but in the scalar multiplication scenario of stage 1 of ECM we can achieve a significant speedup by first converting to Montgomery form, carrying out the (large number of) group operations, then finally converting the result back from Montgomery representation to a number mod N . In doing this, we will have replaced a large number of integer divisions with a corresponding number of bit shifts - a much faster operation, together with two multiplications and one or possibly two additions each (see how below).

We first precompute the least power l of 2 not less than N ,

$$2^{l-1} < N \leq 2^l$$

To convert an integer $r \pmod{N}$ to its Montgomery form $\bar{r}$, we compute

$$\bar{r} \equiv r 2^l \pmod{N}$$

Now, numbers $\bar{a}$ and $\bar{b}$ in Montgomery form can be added, subtracted and multiplied (with reduction), while preserving the ability to convert back to the corresponding residue mod N . This is because there is a ring homomorphism at work. We prove this in theorem 2 below.

In the case of addition and subtraction of numbers in Montgomery form, at most one extra

addition or subtraction (of N) is required to reduce to a number between 0 and $N - 1$. In the case of multiplication of two integers $\bar{a}$ and $\bar{b}$ in Montgomery form, the product is reduced using the algorithm described below, and the result will be the Montgomery representation $\overline{ab}$ of the product of the corresponding integers a and $b \bmod N$.

The algorithm to compute the Montgomery reduction of a product of numbers in Montgomery representation is referred to as the REDC algorithm. It works as follows.

We precompute the negative inverse of $N \bmod 2^l$,

$$N' \equiv -N^{-1} \bmod 2^l$$

Say we want to compute $\overline{ab}$, the Montgomery representation of ab , where a and b are integers $\bmod N$, and $\bar{a}$ and $\bar{b}$ their respective Montgomery representations. Then, compute

$$x = \bar{a}\bar{b}$$

(Note this is not the same as $\overline{ab}$. Here, $\bar{a}\bar{b}$ is just a product of two integers).

Then, let

$$u \equiv xN' \pmod{2^l}$$

(we keep the l least significant bits of the integer xN') and

$$\bar{c} = (x + Nu)/2^l \tag{2.1}$$

Then either $0 \leq \bar{c} < N$ or $N \leq \bar{c} < 2N$. If $\bar{c} \geq N$ then let $\bar{c} = \bar{c} - N$. Then

$$\bar{c} = c2^l \pmod{N}$$

is the Montgomery representation of $c \equiv ab \pmod{N}$.

Dwelling momentarily on equation 2.1, which is an exact integer division, we may wonder how we know $(x + Nu)$ is divisible by 2^l . It follows because $Nu = xNN'$, and since $NN' \equiv -1 \pmod{2^l}$ by construction, $(x + Nu) \equiv (x - x) \equiv 0 \pmod{2^l}$.

Theorem 2. *Define the map*

$$\begin{aligned} \psi : \mathbb{Z}/N\mathbb{Z} &\rightarrow \mathbb{Z}/N\mathbb{Z} \\ r &\mapsto r2^l \pmod{N} \end{aligned}$$

where addition and multiplication $\bmod N$ are defined as usual, except for the codomain the multiplicative operation is multiplication with ‘REDC’ reduction (as described above). Then ψ is a ring homomorphism.

Proof.

$$\begin{aligned}\psi(0) &\equiv 0 \cdot 2^l & (\text{mod } N) \\ &\equiv 0 & (\text{mod } N)\end{aligned}$$

and

$$\begin{aligned}\psi(a+b) &\equiv (a+b)2^l & (\text{mod } N) \\ &\equiv a2^l + b2^l & (\text{mod } N) \\ &\equiv \psi(a) + \psi(b) & (\text{mod } N)\end{aligned}$$

To show how ψ as defined above preserves REDC multiplication, we must show that

$$\psi(ab) \equiv \psi(a) * \psi(b) (\text{mod } N)$$

where the operation $*$ is multiplication of Montgomery residues with REDC reduction. To see this,

$$\begin{aligned}\psi(a) * \psi(b) &= (a2^l b2^l + a2^l b2^l NN')/2^l \\ &= a2^l b + a2^l b NN' \\ &\equiv ab2^l & (\text{mod } N) \\ &= \psi(ab)\end{aligned}$$

Therefore, ψ is a ring homomorphism. □

2.1.5 Windowing

The objective here is to minimize the number of elliptic curve additions necessary to compute a scalar multiple S of a point P on an elliptic curve. Whereas the scalar multiple S cannot change, our method of computing $[S]P$ can. The double-and-add algorithm was described in section 2.1.2. An alternative is to allow subtractions as well as additions and doublings. The resulting ‘addition-subtraction chains’ can lead to a computation needing comparatively few additions and subtractions. A certain amount of precomputation is required. See [BL08], Section 3 for a detailed description of the state of the art in windowing methods.

2.1.6 Curves with Small Parameters

Together with the faster arithmetic possible with Edwards and Hessian curves, this section describes another speedup which is of central importance in this thesis. As we have seen, ECM involves computing a large scalar multiple of a fixed point P , using a fixed curve E . If the point P /curve E have very small coordinates/parameters then there is another saving to be made - each individual multiplication mod N takes a fraction of the time needed for the multiplication of two integers of size N - in fact we can see the speed of

such multiplications, of a small integer by an integer of size N , approaching $\mathcal{O}(N)$ as opposed to $\mathcal{O}(N^2)$. We can thus effectively ‘eliminate’ modular multiplications from our operation counts in the elliptic curve group operation. See section 3.8 for a discovery that was made in this thesis that makes use of small parameters in the case of twisted Hessian curves, allowing particularly fast addition.

2.1.7 An Obstruction

It would be nice to be able to combine the speed-ups due to Montgomery multiplication and the use of small parameters, but unfortunately this does not seem possible. The key difficulty is that converting to Montgomery form mod N will almost surely render the resulting integer to be of the same size as N - ‘smallness’ of parameters is lost from the start. That is more or less the end of the story - an attempt to carry out multiplications combining integers mod N and integers in Montgomery form did not yield much, apart from ruling out one approach to it.

This meant that a choice had to be made between using small parameters, and using Montgomery arithmetic. It was decided that since an implementation of ECM using Edwards/Hessian curves using Montgomery multiplication would be too similar to an implementation that already exists (GMP-ECM), just with a different curve type, small parameters were to be preferred, in the hope that the combination of Edwards/Hessian curves and small parameters would be competitive. We at least know that EECM-MPFQ, the leading Edwards curve implementation, is competitive with GMP-ECM and is in fact faster in certain cases. The hope was that the added advantage of the quartic torsion improvement (see section 3.6) might give some kind of edge over EECM-MPFQ.

2.2 Stage 2

As previously mentioned, Lenstra's original formulation of factoring using elliptic curves has become known as 'stage 1'. Subsequently, 'stage 2' was developed, which makes use of the stage 1 computation. It succeeds even if the group order $\#E(\mathbb{F}_p)$ fails to be sufficiently smooth, as long as $\#E(\mathbb{F}_p)$ is only short of being B_1 -smooth (for a stage 1 bound B_1) by just one larger prime factor of size at most B_2 . Much of this section draws on the descriptions given in [ZD06].

2.2.1 Naive Method

Stage 2 of ECM aims to improve the success probability of factoring with a given stage 1 bound.

In stage 1, we compute

$$Q = [S]P$$

In stage 2, we compute

$$[\pi]Q$$

for all primes

$$B_1 < \pi \leq B_2$$

in the hope that

$$[\pi]Q = \mathcal{O},$$

for one of the primes π . Here, $\mathcal{O}$ is the identity element in the group, B_1 is the stage 1 bound (i.e. covering all prime powers up to B_1 dividing the group order), P is a point on an elliptic curve, S is the stage 1 scalar multiple, and B_2 is the stage 2 bound, for some integer larger than B_1 . It is hoped that the group order $\#E(\mathbb{F}_p)$ is B_1 -smooth except possibly for one extra prime π larger than B_1 .

Thus, after stage 2, as long as $[\pi]Q = \mathcal{O}$ in $\mathbb{F}_p$ and $[\pi]Q \neq \mathcal{O}$ in $\mathbb{F}_q$, where $N = pq$, we can recover $p = \gcd(N, X)$, where X is the projective X coordinate of $[\pi]Q$ in $\mathbb{Z}/N\mathbb{Z}$. This is because if $[\pi]Q = \mathcal{O} \equiv (0 : 1 : 0) \pmod{p}$, then $X \equiv 0 \pmod{p}$ and N share the factor p .

Since there are $\mathcal{O}(B_2/\log B_2)$ primes in the interval $(B_1, B_2]$, the complexity of the naive stage 2 is $\mathcal{O}(B_2)$.

It is estimated that stage 2 saves a logarithmic factor in the speed of ECM - equal to the log of the stage 2 bound B_2 . This is absorbed in the complexity analysis, meaning the complexity itself is not altered by stage 2, but in practice time is saved because curves become more effective. So the question becomes, 'How large can we make B_2 ?'.

An important initial realisation is that we cannot 'cascade' stage 2 - in other words, hope

to somehow create a stage 3 which makes use of the stage 2 residue, or a stage 4 - in fact that would just be extending stage 1, which slows down significantly as B_1 is increased by orders of magnitude. This highlights the sequentiality of stage 1 - each computation depends on the previous one. The utility of stage 2 allows a much larger range of primes to be covered than would be possible to cover in stage 1 (using various techniques to be outlined below), but for only one extra prime. In practice, looking at actual group order factorizations, it is possible to appreciate some of the improvement in success probability. Certainly more group order factorizations are covered for a given B_1 than those that are merely B_1 - smooth.

2.2.2 Birthday Paradox Continuation

Recall the Birthday Paradox, that there are two people in a random group of 23 who likely share the same birthday. This is because

$$\left(1 - \frac{1}{365}\right) \cdot \left(1 - \frac{2}{365}\right) \cdot \left(1 - \frac{3}{365}\right) \cdots \left(1 - \frac{23-1}{365}\right) < 0.5$$

since by first selecting one person in the group, there is a 1 in 365 chance that a second person shares a birthday with the first (therefore a probability of $1 - 1/365$ that they have different birthdays), then a third person has a 2 in 365 chance of sharing a birthday with either of the first two (therefore a probability of $1 - 2/365$ that they all have different birthdays) and so on, and since we require the first two to have different birthdays, AND the first three to have different birthdays, AND the first four... means we must multiply the probabilities, we find that a group of 23 people is the minimum size that this probability, that all in the group have different birthdays, is less than 0.5, or in other words, the probability of the complement of this, that there are at least two in any group of 23 who share a birthday, is greater than 0.5.

(Note we can see from the above that we need 366 people to be *certain* that at least two share a birthday, since the probability of the complement includes the factor $1 - (366 - 1)/365 = 0$, which agrees with the obvious.)

This generalizes for any N , where the probabilities are uniformly distributed. We arrive at the formula

$$P = 1 - \frac{(N-1)!}{N^{k-1}(N-k)!}$$

for the probability that selecting k elements from N will result in a match.

It is possible to show, using calculus, given k and N , that a lower bound for P is

$$P \geq 1 - e^{-\frac{k(k-1)}{2N}}.$$

Thus, we can see that if $k \gg \sqrt{2N}$, then P will be close to 1.

Remark 8. This phenomenon can be used to speed up stage 2 of ECM. Instead of matching birthdays, we have a pair from a set of size $\mathcal{O}(B_2)$ that sum to the elliptic curve identity element. Thus, as we pick more than $\sqrt{B_2}$ pairs of multiples of the stage 1 residue Q , with the multiples selected at random, it becomes increasingly likely that some pair will add to the identity in $E(\mathbb{F}_p)$, allowing us to recover p .

2.2.3 FFT Continuation

The main drawback of the Birthday Paradox continuation is that it is probabilistic, and although likely, is not guaranteed to find a stage 2 factor, even if the stage 2 condition is met. In fact, it is possible to account for all primes in the stage 2 range deterministically, in $\mathcal{O}(\sqrt{B_2}(\log \sqrt{B_2})^{2+o(1)})$ time using $\mathcal{O}(\sqrt{B_2} \log \sqrt{B_2})$ space, thanks to the method of Peter L. Montgomery [Mon92], which has become known as the FFT continuation, because of its reliance on FFT speedup.

We will describe how the FFT continuation works below, introducing along the way an improvement that reduces the total computation needed, and show how the FFT continuation achieves the complexity mentioned above.

The first observation is that it is possible to write any prime $B_1 < \pi < B_2$ as

$$\pi = i \cdot d_1 + j$$

where $d_1 = \mathcal{O}(\sqrt{B_2})$, $0 \leq i \leq B_2/d_1 = \mathcal{O}(\sqrt{B_2})$ and $0 < j < d_1$. Also note that if we were to consider all i and j in these ranges, it would cover more numbers than just the primes in the stage 2 range (indeed all numbers) - but this is not a disadvantage if overall we obtain better than an $\mathcal{O}(B_2)$ algorithm.

Next, define a rational function $\psi : E(\mathbb{Q}) \rightarrow \mathbb{Q}$ with the property that for any $P \in E(\mathbb{Q})$, $\psi(P) = \psi(-P)$. We will see why below. For example, if we have a curve in short Weierstrass form we may take

$$\psi(P) = x,$$

the x coordinate of a point $P = (x, y)$ in affine coordinates.

Now, if $[\pi]Q = [i \cdot d_1 + j]Q = \mathcal{O} = (0 : 1 : 0) \pmod{p}$ for $p|N$, then

$$[i \cdot d_1]Q = -[j]Q$$

and therefore

$$\psi([i \cdot d_1]Q) = \psi([j]Q)$$

and in particular

$$\psi([i \cdot d_1]Q) - \psi([j]Q) \equiv 0 \pmod{p}$$

So, we compute

$$R = \prod_{i=0}^{\lfloor \frac{B_2}{d_1} \rfloor} \prod_{\substack{j=1 \\ (j, d_1)=1}}^{d_1-1} (\psi([i \cdot d_1]Q) - \psi([j]Q)) \pmod{N} \quad (2.2)$$

(note that since we are considering primes π we may restrict j to values coprime to d_1). We will now show the idea of the FFT continuation. We can construct a polynomial from the above product, where we replace $\psi([i \cdot d_1]Q)$ for all i by the unknown x , and allow $\psi([j]Q)$ to vary as before. Thus we obtain the following polynomial:

$$G(x) = \prod_{\substack{j=1 \\ (j, d_1)=1}}^{d_1-1} (x - \psi_j) \pmod{N}$$

where $\psi_j = \psi([j]Q)$. We have that $G(x) \in \mathbb{Z}/N\mathbb{Z}[x]$.

This polynomial can be constructed in $\mathcal{O}(\sqrt{B_2} \log \sqrt{B_2})$ time and space using the ‘product tree’ algorithm for polynomials (see section 2.2.5). Now, there is a task known as ‘multipoint polynomial evaluation’ (see section 2.2.6), i.e. to evaluate a given polynomial at many values, which has been studied and can be done in $\mathcal{O}(n(\log n)^{2+o(1)})$ time and $\mathcal{O}(n \log n)$ space, where n is the degree of the polynomial and $n+1$ the number of evaluations, using the ‘remainder tree’ or improved ‘scaled remainder tree’ algorithm (the latter of which is due to Daniel J. Bernstein). Where this becomes relevant is that if we evaluate $G(x)$ at each of $\psi([i \cdot d_1]Q)$ for $i = 0, 1, \dots, \lfloor B_2/d_1 \rfloor$, and multiply all of these values together $\pmod{N}$, we have exactly computed the product R of (2.2) - so we may then compute

$$\gcd(R, N)$$

and be certain that if the stage 2 condition is satisfied, it will recover a non-trivial factor of N . Where the FFT comes into play is in speeding the computation of the (very large) polynomial $G(x)$ via the product tree. Thus, the announced time and space complexity of stage 2 is met by the use of the remainder tree algorithm for multipoint polynomial evaluation.

We now show how the number of j values that we need to consider can be reduced

significantly. Recall that

$$\psi(P) = \psi(-P)$$

(Note that in the Weierstrass case, as mentioned, we can take the x -coordinate of P - this has an analogue for other curve shapes, for example a point on an Edwards curve has the same y coordinate as its negative - this can be worked out for each type of curve).

Keeping in mind that we ultimately want to account only for i and j that give a prime π , we note that if $\pi = i \cdot d_1 + j$ is prime, then

$$\gcd(j, i \cdot d_1) = 1$$

Since it is fixed, we have more control over d_1 than over i and j . But if we vary d_1 a little from the obvious choice $\lfloor \sqrt{B_2} \rfloor$, for example by ensuring that both 2 and 3 divide d_1 (i.e. that 6 divides d_1), we find that we have certain redundant j values - now we no longer need to consider any j that is a multiple of 2 or 3 (because then $i \cdot d_1 + j$ would not be prime). So we only need to consider j that are congruent to 1 (mod 2) or congruent to 1, 2 (mod 3). Put another way, we only need to consider $j \equiv 1, 5 \pmod{6}$. Not only that, but since $\psi([j]Q) = \psi(-[j]Q)$, if we are working mod 6 we find that $-5 \equiv 1 \pmod{6}$, so in fact we only need to consider $j \equiv 1 \pmod{6}$, i.e. only 1 in every 6 values of j .

We can improve on this by requiring that 2, 3 and 5 divide d_1 . Now we assume 30 divides d_1 and we include only the ‘positive’ residues mod 30 that are coprime to 30, which are 1, 7, 11 and 13. So now we only need to consider 4 in every 30 values of j , i.e. about 1 in every 7.5 values of j , an improvement on 1 in every 6. This might be a good place to stop, since also requiring 7 to divide d_1 only brings a slight improvement while many more cases need to be considered, possibly making the code of an implementation less readable, as well as more severely restricting the values of d_1 that may be chosen.

We should note that in having to consider fewer j values, we are reducing the number of elliptic curve computations that need to be done in stage 2. Also, it would be inefficient if for each value of j , we had to compute $[j]Q$ from scratch. A better way is to precompute $j_1 = Q$, $j_7 = [7]Q$, $j_{11} = [11]Q$, $j_{13} = [13]Q$ and $j_{30} = [30]Q$, whereupon adding j_{30} to each of j_1 , j_7 , j_{11} and j_{13} successively allows us to cover the positive residues of $j \pmod{30}$, until we arrive at $j = d_1$ or greater.

An important observation is that the successive additions above are done only to ensure all required j values are covered, in fact more are covered than required. The only j values for which we keep $\psi([j]Q)$ are precisely those for which $\gcd(j, d_1) = 1$, and there are

$$\frac{\phi(d_1)}{2}$$

of these up to sign, where ϕ is the Euler totient function. This means we have more control in the latter part of the FFT continuation, where $\phi(d_1)/2$ will become the degree

of the (large) polynomial $G(x)$. Also, we can reduce the degree of $G(x)$ (which is desirable), without compromising the size of B_2 , by requiring that d_1 be composed of only tiny primes, since this minimizes the totient.

Remark 9. Software that uses the standard continuation has a distinct advantage over implementations of ECM that use the naive stage 2 method. Note that a significant part of the remainder tree algorithm involves multiplying polynomials and it turns out that a significant part of multiplying polynomials efficiently is multiplying large integers efficiently (see the next section). The integers involved are of the size where Schönhage-Strassen (i.e. FFT) multiplication is more efficient than schoolbook multiplication. So whereas part of the advantage of the standard continuation is due to the improved $\mathcal{O}(\sqrt{B_2} (\log \sqrt{B_2})^{2+o(1)})$ complexity, there is yet another gain to be made in speeding up the remainder tree polynomial multiplications themselves. It is mentioned in [BBLP13], section 5.7, that GMP-ECM outperforms EECM-MPFQ in stage 2 for $d_1 = 510510$, probably because GMP-ECM implements polynomial multiplication using FFT integer multiplication (which is automatic in GMP integer arithmetic past a certain threshold), while EECM-MPFQ uses the NTL library for polynomial multiplication, which it is mentioned does not have ‘FFT-based-polynomial-arithmetic optimizations’.

2.2.4 Kronecker-Schönhage Polynomial Multiplication

As mentioned above, multiplication of polynomials features prominently in the FFT continuation for stage 2 of ECM. These polynomials usually have large degree (e.g. 100,000), and each coefficient is an integer in the ring $\mathbb{Z}/N\mathbb{Z}$, where N is the number to be factored. To multiply two such polynomials, we might appreciate that an algorithm similar to the schoolbook algorithm for integer multiplication could be used, and with time complexity $\mathcal{O}(d^2)$, where d is the maximum degree of the polynomials.

However this can be improved upon, first using an idea attributed to Kronecker. We transform the multiplication of two polynomials

$$f(x) = f_0 + f_1x + f_2x^2 + \cdots + f_dx^d$$

and

$$g(x) = g_0 + g_1x + g_2x^2 + \cdots + g_dx^d$$

to the multiplication of the two related integers

$$F = f_0 + f_12^l + f_22^{2l} + \cdots + f_d2^{dl}$$

and

$$G = g_0 + g_12^l + g_22^{2l} + \cdots + g_d2^{dl}$$

where l is the minimum number of bits required to store any integer in the product poly-

nomial.

We can see how each coefficient ‘resides’ in a block of l bits, and we ensure that the blocks do not overlap by letting l be such that any possible coefficient in the product polynomial is less than 2^l . The maximum coefficient size in the product, before it is reduced mod N , is dN^2 . So we have

$$2^{l-1} < dN^2 \leq 2^l$$

Then, we just multiply the two integers F and G together, and if we use Schönhage-Strassen multiplication, this takes time $\mathcal{O}(d \log d \log \log d)$ (we have absorbed N , here a constant). To recover the coefficients of the product, we scan FG one block of l bits at a time, extract the block, and reduce it mod N .

2.2.5 Polynomial Product Tree

We could envisage a simple method to compute the product of polynomials $f_1, f_2, \dots, f_n$, each of degree d - simply multiply f_1 by f_2 , then f_3 by $f_1 f_2$, then f_4 by $f_1 f_2 f_3$ and so on, until we have used all polynomials. The problem with this approach is that each successive multiplication is of a polynomial of degree d by a polynomial of rapidly increasing degree, with an extra d terms each iteration. This might not sound like much, but it very quickly becomes unwieldy, particularly if d is large. We can derive the complexity of this method. The total cost of polynomial multiplications is proportional to

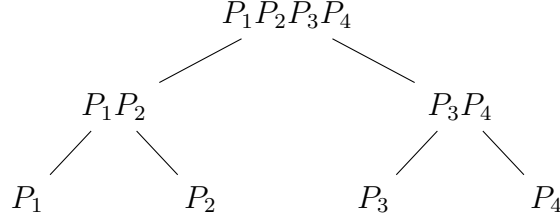
$$d \cdot d + d \cdot 2d + d \cdot 3d + \dots + d \cdot (n-1)d$$

which is equal to

$$\begin{aligned} & d^2 (1 + 2 + 3 + \dots + (n-1)) \\ &= d^2 (n-1)(n-2)/2 \\ &= \mathcal{O}(n^2) \end{aligned}$$

since d is fixed with respect to n . A much better approach is to use the product tree algorithm.

The product tree algorithm is a ‘divide and conquer’ algorithm. For simplicity assume that n , the number of polynomials to be multiplied, is a power of 2. We group the polynomials in pairs, then multiply the two in each pair giving $n/2$ polynomials of degree $2d$. Then we group these in pairs, and multiply each of a pair, giving $n/4$ polynomials of degree $4d$. We continue in this way until we are left with 2 polynomials, of degree $nd/2$, which we multiply to give the final product, of degree nd . There are $n/2^k$ multiplications of polynomials of degree $2^k d$ at each level $k = 1, 2, \dots, \log_2 n$, and since the amount of work in each level is conserved at $\mathcal{O}(n)$, we have done $\mathcal{O}(n \log n)$ work in total in computing the entire tree. This approach can also be used to multiply large numbers of integers together, as is necessary in the computation of the stage 1 scalar multiple of ECM (see section 2.1.2).



2.2.6 Bernstein Scaled Remainder Tree

In implementing the standard continuation for stage 2 of ECM, it is necessary to evaluate a polynomial of degree $d = \phi(d_1)$ at $d+1$ values, where d_1 is a stage 2 parameter. This can be done using the ‘remainder tree’ algorithm, which is described below, before describing Bernstein’s improvement.

We first need a theorem from abstract algebra (see, for example, [Nic07]).

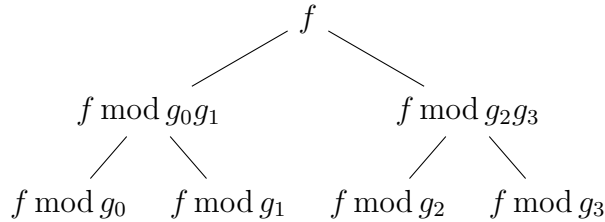
Theorem 3 *Remainder Theorem.* *If R is a commutative ring and $f(x) \in R[x]$, then $f(a) \equiv f(x) \pmod{\langle x - a \rangle}$.*

Proof. Suppose $f(x) = \sum_k f_k x^k$. Then,

$$\begin{aligned}
 f(x) - f(a) &= \sum_k f_k (x^k - a^k) \\
 &= (x - a) \sum_k f_k (x^{k-1} + \dots + a^{k-1}) \\
 &= (x - a)g(x) \\
 \implies f(a) &= f(x) - (x - a)g(x) \\
 \implies f(a) &\equiv f(x) \pmod{\langle x - a \rangle}
 \end{aligned}$$

□

Suppose we want to evaluate a polynomial f of degree d at $d+1$ values $f_0, f_1, \dots, f_d$. Consider an illustrative example where $d = 3$. We construct the remainder tree as follows (note that in this case $d+1$ is a power of two, which is the simplest case and leads to a perfect binary tree).

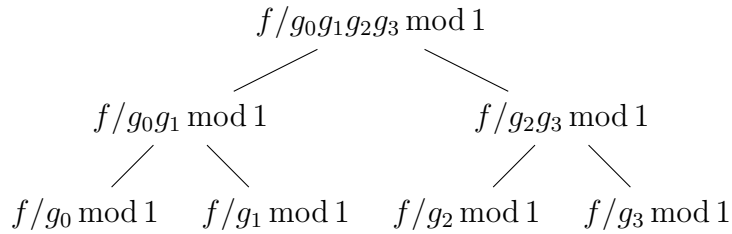


Here, $g_0 = x - f_0$, $g_1 = x - f_1$, $g_2 = x - f_2$ and $g_3 = x - f_3$.

To compute the remainder tree, we proceed from the top node down. At the second level, we reduce $f \bmod g_0g_1$ and $f \bmod g_2g_3$, where g_0g_1 is the product of g_0 and g_1 , g_2g_3 the product of g_2 and g_3 . At each successive level, we take the result of the previous node and reduce it modulo some product of the degree-1 polynomials g_0, g_1, g_2, g_3 . We can now see how polynomial multiplication becomes a significant part of the process, and why the required memory is $\mathcal{O}(d \log d)$. The sum of the degrees of the polynomials at each level is conserved at d , since the degree of f is d , in the next level there are 2 polynomials of degree $d/2$, and so on, and where (at least in a perfect binary tree) there are $\log_2 d$ levels apart from the root. In fact it is necessary to precompute the product tree of g_0, g_1, g_2, g_3 , and store it in memory at once, because in the course of traversing the remainder tree, the intermediate products must be retrieved in a way that is probably difficult to linearize.

One difficulty with the remainder tree is that at every node in the tree, a polynomial must be reduced modulo another, which is basically the same as a polynomial division with remainder. The drawback of this is analogous to that of having to compute many integer modular reductions - division is slower in both cases than multiplication.

This is where Bernstein's 'scaled remainder tree' [Ber] makes a practical improvement, by replacing all but one polynomial division by polynomial multiplications. Illustrating again by way of an example, using the same polynomial and values as before for the sake of simplicity, we have the tree



The root node here is the first d terms of the Laurent series expansion of the quotient of the polynomials f and $g_0g_1g_2g_3$. To compute this in practice, we first compute the 'precision- d reciprocal' $(g_0g_1g_2g_3)^{-1}$, using the method outlined (for example) in the next section. We then multiply the polynomials f and $(g_0g_1g_2g_3)^{-1}$ and keep the d most significant coefficients. A calculation of this type is done only once, and is equivalent to a precision- d division. After that, at each node, we multiply the previous node by a certain product of g_0, g_1, g_2, g_3 . For example, to obtain the nodes on the second level, multiply the root node by g_2g_3 at the left, and multiply the root node by g_0g_1 at the right. Say the product polynomial at a node is h , and the degree of the previous node is r . There is one extra step for each node - we reduce the result of the multiplication modulo the polynomial $x^{r-\deg h} - 1$. However, this is not a difficult division, in fact we don't really think of it as a division, since all we have to do is keep $r - \deg h$ coefficients, and any coefficients of higher terms are added to the terms they 'wrap around' to modulo $x^{r-\deg h} - 1$. Then, by the time we arrive at the leaf nodes, $r - \deg h$ will equal zero, the reduced polynomial will

be a polynomial of degree zero, i.e. an element of the ring $\mathbb{Z}/N\mathbb{Z}$. These are precisely the values of f evaluated at $f_0, f_1, f_2, \dots, f_d$.

Remark 10. It is not too difficult to adapt the above to create an ‘unbalanced’ scaled remainder tree, i.e. a scaled remainder tree that is not a perfect binary tree, for example if we need to evaluate a polynomial of degree d at $d + 1$ values, where $d + 1$ is not a power of two (in fact this was done in the implementation to give more control over the possible choices of d_1). A key observation is that the above method works almost verbatim - we just have to ensure that if at any node, $r - \deg h = 0$, then we recognize that we are at a leaf node and the most significant coefficient of the product will be a polyeval value. However, doing this means that we may not be able to use the ‘ $x^d + 1$ ’ trick to speed polynomial multiplication - see remark 20 in section 5.4 - unless we use zero padding.

Algorithm 1 Unbalanced Scaled Remainder Tree

Input:

Monic polynomial $F \in \mathbb{Z}/N\mathbb{Z}[x]$ of degree d and $d + 1$ values $x_0, x_1, \dots, x_d \in \mathbb{Z}/N\mathbb{Z}$

Output:

Product $F(x_0) \cdot F(x_1) \cdots F(x_d) \pmod N$

```

1: procedure SCALEDREMTREE
2:    $product \leftarrow 1$ 
3:    $nodePQ \leftarrow \deg(F)$  most significant terms of H/F
4:   push  $nodePQ$  to stack
5:   while stack is non-empty do
6:      $nodePQ \leftarrow stack.pop$ 
7:      $P \leftarrow$  left child node of position  $nodePQ$  in product tree used to compute F
8:      $nodeP \leftarrow nodePQ \times P \pmod{(x^{\deg(nodePQ)+1} - 1)}$ 
9:     if  $\deg(nodePQ) = \deg(P)$  then
10:       $product \leftarrow product \times \deg(P)th$  coefficient of  $nodeP \pmod N$ 
11:    else
12:       $d \leftarrow \deg(nodePQ) - \deg(P)$ 
13:       $newnode \leftarrow d$  most significant terms of  $nodeP$ 
14:      push  $newnode$  to stack
15:    end if
16:     $Q \leftarrow$  right child node of position  $nodePQ$  in product tree used to compute F
17:     $nodeQ \leftarrow nodePQ \times Q \pmod{(x^{\deg(nodePQ)+1} - 1)}$ 
18:    if  $\deg(nodePQ) = \deg(Q)$  then
19:       $product \leftarrow product \times \deg(Q)th$  coefficient of  $nodeQ \pmod N$ 
20:    else
21:       $d \leftarrow \deg(nodePQ) - \deg(Q)$ 
22:       $newnode \leftarrow d$  most significant terms of  $nodeQ$ 
23:      push  $newnode$  to stack
24:    end if
25:  end while
26:  return  $product$ 
27: end procedure

```

2.2.7 Polynomial Pseudo-Inverse (Reciprocal)

A simple method to compute the ‘precision- d ’ reciprocal of a polynomial f of degree d with coefficients in $\mathbb{Z}/N\mathbb{Z}$, where $f(0) = 1$, is to use Newton’s method. It uses $\mathcal{O}(d \log d \log \log d)$ operations in the ring. Some redundancies in this method were identified in [Ber04], where a faster algorithm (by a constant factor) is described. In the implementation (see section 4) it turned out that Newton’s method was acceptably fast, so these extra optimizations were not done.

Newton’s method is described in [Ber08], Section 6. Here we describe an equivalent non-recursive method.

Algorithm 2 Polynomial Reciprocal to d terms of accuracy

Input:

Monic polynomial $f \in \mathbb{Z}/N\mathbb{Z}[x]$ of degree d

Output:

Polynomial $1/f$ to d terms of accuracy

```

1: procedure POLYINV
2:    $k \leftarrow 1$ 
3:    $l \leftarrow \lceil \log_2 d \rceil$ 
4:    $g_k \leftarrow 1$ 
5:   iterate:
6:      $m \leftarrow \lceil d/2^{l-k} \rceil$ 
7:      $f_m \leftarrow f \bmod x^m$ 
8:      $g_{k+1} \leftarrow g_k - (f_m g_k - 1) g_k \bmod x^m$ 
9:     if  $k < l$  then
10:        $k \leftarrow k + 1$ 
11:       goto iterate
12:     end if
13:     output  $g_{k+1}$ .
14: end procedure
```

Remark 11. Note that computing $f_m = f \bmod x^m$ is easy - we keep the first m coefficients of f .

We now prove the correctness of algorithm 2 (note that this proof was not found elsewhere in the literature).

Theorem 4. *Algorithm 2 computes the precision- d reciprocal of $f \in \mathbb{Z}/N\mathbb{Z}[x]$ of degree d with $f(0) = 1$ for any $d \geq 1$.*

Proof. Note the identity

$$1/f - (g_k - (f g_k - 1)g_k) = f(1/f - g_k)^2 \quad (2.3)$$

which holds for any g_k . If $1/f$ happens to be equal to g_k for some k modulo some x^m , we can say more - in fact then g_{k+1} , as computed by algorithm 2, will equal $1/f$ modulo any power of x up to x^{2m} . To see this, note that $1/f - g_k$ in the RHS of the identity 2.3 is

a multiple of x^m (since by assumption $1/f \equiv g_k \pmod{x^m}$). Therefore the RHS, and hence the LHS, is a multiple of x^{2m} , i.e.

$$1/f - (g_k - (fg_k - 1)g_k) \equiv 0 \pmod{x^{2m}}$$

or

$$1/f \equiv (g_k - (fg_k - 1)g_k) \pmod{x^{2m}}.$$

Now, $1/f \equiv 1 \pmod{x}$ since $f(0) = 1$. So by the above argument, taking $g_k = 1$ and $m = 1$, we have $1/f \equiv g_{k+1} \pmod{x^{2m}}$. Now we can see that if m at most doubles on each iteration, the new polynomial g_{k+1} will equal $1/f \pmod{x^m}$ at every iteration. So now we show that m at most doubles on each iteration. But this is clear since $m = \lceil d/2^{l-k} \rceil$, where d and l are fixed and k increases by only 1 each iteration.

□

2.2.8 Polynomial Modulo Reduction

A necessary task in the algorithm of the next section (Stage 2 Blocks) is to reduce one (large) polynomial modulo another, so here we give an algorithm to do this, described in [Ber08], section 17. The modulus polynomial must be monic, but for example if $f \in \mathbb{Z}/N\mathbb{Z}[x]$ we can make it monic by dividing through by the leading coefficient. However, this might not be necessary in practice as the polynomial might be monic by construction (as it is for the purpose of the algorithm of the next section).

Algorithm 3 Polynomial Modulo Reduction - compute $G \bmod F$

Input:

Polynomials $F, G \in \mathbb{Z}/N\mathbb{Z}[x]$, with F monic.

Output:

$G \bmod F$

- 1: **procedure** POLYREDUCE
 - 2: $F \leftarrow$ polynomial modulus
 - 3: $1/F \leftarrow$ reciprocal of F
 - 4: $G \leftarrow$ polynomial to reduce modulo F
 - 5: $G_{rev} \leftarrow$ reverse coefficients of G
 - 6: $H \leftarrow G_{rev} \cdot 1/F$
 - 7: $q \leftarrow H \bmod x^{\deg H - \deg F + 1}$
 - 8: (i.e. discard all powers of x in H not less than $\deg H - \deg F + 1$)
 - 9: Reverse q
 - 10: $G \leftarrow$ re-reverse coefficients of G_{rev}
 - 11: $r \leftarrow G - q \cdot F$
 - 12: output r
 - 13: **end procedure**
-

2.2.9 Stage 2 Blocks

As mentioned in section 2.2.3, the space complexity of the standard continuation, for a stage 2 bound B_2 , is $\mathcal{O}(\sqrt{B_2} \log \sqrt{B_2})$. It turns out that the limiting factor in using the standard continuation is the memory requirement, which imposes a practical limit on the number of digits in B_2 of not more than 14 or 15 digits, at the higher end requiring gigabytes of storage. A bound of this size can still be used with reduced memory requirements using ‘stage 2 blocks’, which can also offer a slight speed improvement for stage 2 under certain conditions.

Algorithm 4 Stage 2 Blocks

Input:

Monic polynomial $F \in \mathbb{Z}/N\mathbb{Z}[x]$ of degree d , number of blocks k , with k polynomials $G_1, G_2, \dots, G_k \in \mathbb{Z}/N\mathbb{Z}[x]$ each of degree $d - 1$

Output:

Polynomial $G_1 \cdot G_2 \cdots G_k \pmod{F}$ evaluated at the roots of F

```

1: procedure STAGE2BLOCKS
2:    $F \leftarrow$  polynomial constructed from ‘baby step’ roots
3:    $k \leftarrow$  number of stage 2 blocks
4:    $\deg G \leftarrow$  degree of block polynomial =  $\deg F - 1$ 
5:    $[S]P \leftarrow$  stage 1 residue
6:    $Q_0 \leftarrow [d_1]([S]P)$ 
7:    $Q \leftarrow Q_0$ 
8:    $H \leftarrow 1$ 
9:   for  $j = 1$  to  $k$  do
10:    for  $i = 1$  to  $\deg G$  do
11:       $i$ th root of  $G_j \leftarrow \psi(Q)$ 
12:       $Q \leftarrow Q + Q_0$ 
13:    end for
14:    Construct  $G_j$  from its roots
15:    if  $j > 1$  then
16:       $H \leftarrow H \cdot G_j \pmod{F}$ 
17:    else
18:       $H = G_1$ 
19:    end if
20:  end for
21:  Evaluate  $H$  at the roots of  $F$ 
22: end procedure

```

The method works by first dividing the stage 2 range into k blocks. In fact it is advisable to adjust (increase) the stage 2 bound to ensure that the blocks are all of the same size (which was done in the implementation). On its own, dividing up the range does not yield a method to reduce the required memory - how are the blocks to be processed? A way to do this that saves computation is not immediately apparent. But there is an elegant way that makes use of the homomorphism theorem. To explain how it is done, we first prove the following theorem. Although possibly not new, this proof was not found elsewhere in the literature.

Theorem 5. Let $F, H \in \mathbb{Z}/p\mathbb{Z}[x]$ with $H = G_1 \cdot G_2 \cdot G_3 \cdots G_k$, where $G_j(a) \equiv 0 \pmod{p}$ for some $a \in \mathbb{Z}/p\mathbb{Z}$, with $F(a) \equiv 0 \pmod{p}$ (i.e. a is a root of F). Let $R(x) = H(x) \bmod F(x)$. Then $R(a) \equiv 0 \pmod{p}$.

Proof. Define the map ϕ_a as follows:

$$\begin{aligned}\phi_a : \mathbb{Z}/p\mathbb{Z}[x] &\rightarrow \mathbb{Z}/p\mathbb{Z} \\ G(x) &\mapsto G(a)\end{aligned}$$

By the Evaluation Theorem (see e.g. Theorem 5, chapter 4 of [Nic07]), ϕ_a is an onto homomorphism. Then, by the homomorphism theorem, we have that $\mathbb{Z}/p\mathbb{Z}[x]/\ker \phi_a \cong \mathbb{Z}/p\mathbb{Z}$. In particular if $R(x) \in \ker \phi_a$ then since $\phi_a(R(x)) \mapsto R(a)$, we must have $R(a) \equiv 0 \pmod{p}$. So it remains to show that $R(x) \in \ker \phi_a$. We have that since $R(x) = H(x) \bmod F(x)$, then $R(x) = H(x) + k(x)F(x)$, for some $k(x) \in \mathbb{Z}/p\mathbb{Z}[x]$. But $H(x) = G_j(x)H'(x)$, where $G_j(a) \equiv 0 \pmod{p}$ by assumption, so

$$R(x) = G_j(x)H'(x) + k(x)F(x)$$

But $x - a$ is a factor not only of $G_j(x)$ but also of $F(x)$ (since a is a root of both $\bmod p$). Therefore

$$R(x) = [x - a]R'(x)$$

(using square brackets to distinguish from ideals generated by a polynomial), i.e. $R(x) \in (x - a)$, the polynomial ideal generated by $x - a$. Finally, we show that $\ker \phi_a = (x - a)$.

We have that if $g(x) \in \ker \phi_a$ then it is sufficient that $g(x) \in (x - a) = k(x)[x - a]$ for $k(x) \in \mathbb{Z}/p\mathbb{Z}[x]$ (where we have again used square brackets to indicate that $[x - a]$ is a polynomial, not an ideal), since $\phi_a(g(x)) \mapsto g(a) = k(a)[a - a] \equiv 0 \pmod{p}$. To show that it is necessary, suppose $\phi_a(g(x)) \equiv 0 \pmod{p}$. But then $g(a) \equiv 0 \pmod{p}$ by definition of ϕ_a , which means that $x - a$ is a factor of $g(x)$ by the factor theorem. Therefore $g(x) = k(x)[x - a]$, i.e. $g(x) \in (x - a)$. □

Now we see how this achieves the memory saving - we arrange for the degree of the G_j to be one k th the size of the stage 2 block of $\mathcal{O}(\sqrt{B_2})$ roots, and $\deg F - 1 = \deg G_j$. All of these polynomials are smaller than would otherwise have been required by a factor of about k . Not only that, but now the polynomial R that we must evaluate at the roots of F is the same size as the G_j , since $R(x) = H(x) \bmod F(x)$. So the end result is that the required POLYEVAL is much smaller, with a corresponding decrease in memory required. Increasing the number of blocks to be greater than 1 can also lead to a time saving. See [ZD06], section 3.3 where it is shown that it is possible to reduce the total cost of the stage 2 POLYEVAL operation from $4qM(d)\log(d)$ to $3.46qM(d)\log(d)$, where $qM(d)\log(d)$ is the cost of a product tree operation (with d the degree of polynomials, and $M(d)$ the cost of polynomial multiplication), provided Bernstein's scaled remainder tree (2.2.6) is used, with $k = 3$ blocks. Even though a time saving may be possible, in practice the usefulness

of the method is more that higher B_2 is achievable with much-reduced memory, even with slightly increased time requirement.

2.2.10 The Brent-Suyama Extension

The Brent-Suyama extension would seem to offer a potential improvement of making stage 2 up to around twice as likely to succeed. Note that this does not equate to a log factor improvement, in fact it is less. The Brent Suyama extension was not done in the implementation (probably detrimentally but not to a large degree).

Observe that either $x + y$ or $x - y$ divides $x^k \pm y^k$ for any $k \geq 2$. This fact can be used to speed up stage 2 of ECM. Instead of just computing roots y and evaluating

$$G(x) = \prod_j (x - y) \pmod{N}$$

at all x , we choose some power k , compute each y^k and evaluate

$$G(x) = \prod_j (x - y^k) \pmod{N}$$

at $x = i^k$, for $i = 1, \dots, i_{max}$. Thus there are possibly many more primes that may be hit in stage 2 (the factors of $x^k \pm y^k$ surplus to $x \pm y$). One complication is having to compute points $[i^k]Q$ instead of $[i]Q$. To do this we may use a table of differences. Illustrating by way of example, if $k = 4$, we construct the table

1	16	81	256	625	1296	2401	...
	15	65	175	369	671	1105	...
		50	110	194	302	434	...
			60	84	108	132	...
				24	24	24	24

Entries correspond to elliptic curve scalar multiples of Q , but we do not compute all the scalar multiples, just the first downward diagonal. The lowest row repeats only 24. Subsequent entries are computing by summing the term to the lower left with the entry to the immediate left (this is an elliptic curve addition). The table is built up in this manner, leaving the fourth power scalar multiples $[i^4]Q$ for $i = 1, 2, 3, \dots$ as the first row. This can be done because the i values are in sequence. We must do approximately k times the work we would have done for $k = 1$.

In practice, it is possible to increase the likelihood of finding extra primes by computing $D_k(i)$ instead of i^k , where $D_k(i)$ is the degree- k ‘‘Dickson polynomial’’. This is defined

recursively as

$$D_1 = x$$

$$D_2 = x^2 + 2$$

$$D_{k+2} = xD_{k+1} + D_k \quad \text{for } k \geq 1.$$

3 Recent Improvements to ECM

Within the past 10 years some incremental improvements to stage 1 have been made, starting with the discovery of ‘Edwards curves’ (named after Harold M. Edwards, who described the new formulation of elliptic curves in [Edw07]), which allow faster point addition and doubling (speeding up the stage 1 scalar multiple computation), with the subsequent suggestion by Dujella/Najman in [DN12] that an improvement in elliptic curve effectiveness is possible, shown using the theory of elliptic curves defined over number fields of small degree (e.g. quadratic/quartic number fields). In fact it was the combination of these two ideas that formed the starting point for the research of the present work.

3.1 Brief Overview of the Theory of Elliptic Curves

3.1.1 History and Definition

A description of the evolution of the theory of elliptic curves is given in [Sti95]. The modern theory of elliptic curves has connections to a number of areas in mathematics, for example algebraic geometry, number theory, analysis, topology. Below is the formal definition of an elliptic curve.

Definition 2. *An elliptic curve over a field K is a projective, nonsingular curve E of genus one with a distinguished point $\mathcal{O} \in E(K)$.*

An elliptic curve has a geometric aspect (it is a curve), and an algebraic aspect, as outlined below. This means elliptic curves are studied as part of algebraic geometry, which provides a setting for elliptic curves and more general algebraic varieties (for example hyperelliptic curves and higher-dimensional varieties). The subject has been built up particularly in the last 100 or so years, from its beginnings in considering curves arising from the study of arc lengths of ellipses (from where the terminology ‘elliptic’ derives, and although an ellipse is a curve, it does not satisfy the defining properties of definition 2, so an ellipse is not an elliptic curve), to the more modern definition given. Elliptic curves can be studied in much depth, and have found applications in number theory and recently in cryptology. A conjectured link between elliptic curves and modular forms (which arise in mathematical analysis), was proved by Andrew Wiles in 1995 [Wil95], famously proving ‘Fermat’s last theorem’, that

$$x^n + y^n \neq z^n$$

for integers x, y, z and for any integer $n > 2$.

Elliptic curves can be used to encrypt data - if we are given a sequence m of up to b bits and an elliptic curve E defined over a finite field of prime order $p > 2^b$, where m is arranged to be, say, the X coordinate of a point Q on E of (prime) order N , we may effectively ‘scramble’ m by computing some large multiple $R = [S]Q$ and outputting the X coordinate of R . The original data may be recovered by computing $[S^{-1} \bmod N]R$, where m will again be the X coordinate. The security of this operation, where S is kept secret, appeals to the difficulty of the ‘Elliptic curve discrete logarithm’ problem (ECDLP), i.e.

recovering S given only Q and $[S]Q$, for which the best known attack takes time of the order of $\sqrt{p}$, which is exponential in $\log p$ (i.e. there is not even a known subexponential attack). The ElGamal cryptosystem is a public key cryptosystem which relies on the difficulty of computing ECDLP. Elliptic curves can be used in cryptanalysis via their use in ECM, if not directly, at least, for example, as a factoring subroutine in the number field sieve.

3.1.2 Algebraic Properties

The set of points on an elliptic curve forms a group. The group law can be defined geometrically (using chords and tangents) or algebraically (as formulas involving a pair of points in affine/projective form, for example).

3.1.3 The Mordell-Weil Theorem

The Mordell-Weil Theorem, stated below, provides an answer to the question, “How do we know we can generate all rational solutions to an elliptic curve equation?” - but knowing the underlying structure of a mathematical object answers many questions.

Theorem 6 Mordell-Weil Theorem. *Let K be a number field, i.e. a finite extension of $\mathbb{Q}$, and let E/K be an elliptic curve defined over K . Then the group $E(K)$ of points on the curve is finitely-generated, i.e. has the structure*

$$E(K) \cong E(K)_{tors} \oplus \mathbb{Z}^r$$

where $E(K)_{tors}$ is a finite group and r is a non-negative integer.

Proof. See e.g. [Sil09], Chapter VIII. □

The group $E(K)_{tors}$ is known as the torsion subgroup of the elliptic curve, and the integer r is known as the rank.

3.1.4 The Lutz-Nagell Theorem

Here we state a theorem due to Lutz and Nagell (proved independently). We will need this in section 3.2.3. In addition, the theorem provides a way to quickly determine torsion points.

Theorem 7. *Let E be given by $y^2 = x^3 + Ax + B$ with $A, B \in \mathbb{Z}$. Let $P = (x, y) \in E(\mathbb{Q})$ be a non-identity point, where the identity is $\mathcal{O} = (0 : 1 : 0)$. Suppose P has finite order. Then $x, y \in \mathbb{Z}$. Furthermore, if $y \neq 0$ then*

$$y^2 \mid 4A^3 + 27B^2.$$

Proof. See [Was08], theorem 8.7. □

3.2 Torsion, Rank and Number Fields

The torsion of an elliptic curve over a field K persists over any extension of K , since the base field is contained within any extension of it. The same is true of the rank. There are more subtle questions that can be asked, for example, “What torsion subgroups are possible over a given number field?”, or more generally, “What torsion subgroups are possible over all number fields of a given degree?”, or, a wider goal of these efforts, “Can the possible torsion subgroups be classified for all number fields?”. An extra nuance is to require the curve itself (coefficients) to be defined over $\mathbb{Q}$, for example (or a particular number field), while varying the field (or field degree). Some questions of these particular types have been answered in certain cases. On the other hand, questions about the behaviour of rank seem to be more elusive. We can see that there is no obvious obstruction to the possibility of the rank increasing if the degree of the number field over which an elliptic curve is defined is increased, and it can happen in practice. An example of this (with much more background in [Eik04]) is the elliptic curve

$$y^2 = x^3 - x + 3$$

which has rank 1 over $\mathbb{Q}$ (with generator $(2, 3)$), and rank 2 over $\mathbb{Q}(\sqrt{3})$ (with generators precisely $(2, 3)$ and $(0, \sqrt{3})$). It is slightly more involved to show these are the only generators).

3.2.1 Mazur’s Theorem

While the Mordell-Weil theorem answers the question of what the configuration of the structure of an elliptic curve group is, it does not say exactly what the possible structures of the torsion subgroup are (nor how to calculate the rank, although this in particular seems to be an even more difficult question). For an elliptic curve defined over $\mathbb{Q}$, the question of what torsion subgroups can arise has been answered completely, by a theorem of Barry Mazur [Maz77], proved in 1978, which verified a previous conjecture of Ogg.

Theorem 8 Mazur’s Torsion Theorem. *Let $E/\mathbb{Q}$ be an elliptic curve defined over $\mathbb{Q}$. Then*

$$E(\mathbb{Q}) \cong E(\mathbb{Q})_{tors} \oplus \mathbb{Z}^r$$

where $E(\mathbb{Q})_{tors}$ is a finite group having one of the following 15 structures:

$$\begin{array}{ll} \mathbb{Z}/m\mathbb{Z} & \text{for } 1 \leq m \leq 12, m \neq 11, \\ \mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/2m\mathbb{Z} & \text{for } 1 \leq m \leq 4. \end{array}$$

Proof. See [Maz77]. □

3.2.2 The Search for a Generalization

As mentioned, the possible torsion structures for an elliptic curve defined over $\mathbb{Q}$ are known. If we try to generalize this to any number field, the picture is still incomplete - the only other complete classification (at the time of writing) is for quadratic number fields, where we have the following theorem, due to Kamienny [Kam92] (building on work of Kenku and Momose [KM88]).

Theorem 9. *Let E/K be an elliptic curve defined over K , a quadratic number field. Then*

$$E(K) \cong E(K)_{tors} \oplus \mathbb{Z}^r$$

where $E(K)_{tors}$ is a finite group having one of the following 26 structures:

$$\begin{array}{ll} \mathbb{Z}/m\mathbb{Z} & \text{for } 1 \leq m \leq 18, m \neq 17, \\ \mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/2m\mathbb{Z} & \text{for } 1 \leq m \leq 6, \\ \mathbb{Z}/3\mathbb{Z} \oplus \mathbb{Z}/3m\mathbb{Z} & \text{for } m \in \{1, 2\}, \\ \mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/4\mathbb{Z}. & \end{array}$$

Proof. See [Kam92].

□

For number fields of higher degree, only partial results are known. Most is known about the cubic and quartic cases, where in both cases ([JKL11a], resp. [JKL11b]) it is known which torsion subgroups can arise infinitely often for non-isomorphic curves. Even for quintic fields, according to the survey [Sut], it is still an open question which torsion subgroups arise infinitely often.

3.2.3 Relevance to ECM

Theorem 10 below is of central importance in this work. It gives the reason that it is desirable to have as large a torsion subgroup as possible for the elliptic curves used in ECM. Although theorem 10 is stated for elliptic curves over $\mathbb{Q}$ (since the proof is short), in fact there is a more general result valid for elliptic curves over any number field. What is shown is that the torsion subgroup of an elliptic curve E ‘injects’ into $E(\mathbb{F}_p)$ for primes p of good reduction (which covers most primes). In other words there exists an injective map between $E(\mathbb{Q})_{tors}$ and $E(\mathbb{F}_p)$ for primes of good reduction. In this case, this means that the order of the torsion subgroup will divide the group order of $E(\mathbb{F}_p)$, bearing in mind that the points of finite order over $\mathbb{Q}$, when reduced mod p , will form a subgroup of the points over $\mathbb{F}_p$. The larger the divisor, the smaller the cofactor that we require to be smooth. This at least increases the success probability of ECM empirically (see [BBLP13], figures 9.1 and 9.2 for comparisons of ECM success probability for varying torsion subgroup size, and section 5.4 of this thesis for similar statistics). See [Sil09], section VII.3,

Proposition 3.1 for a more general proof of what follows.

Theorem 10. *Let $E : Y^2Z = X^3 + AXZ^2 + BZ^3$ be an elliptic curve with $A, B \in \mathbb{Z}$. Denote the identity element by $\mathcal{O} = (0 : 1 : 0)$. Let p be a prime of good reduction, i.e. $p \nmid 4A^3 + 27B^2$. Let*

$$\begin{aligned} \rho_p : \mathbb{P}^2(\mathbb{Q}) &\rightarrow \mathbb{P}^2(\mathbb{F}_p) \\ (X : Y : Z) &\mapsto (X : Y : Z) \bmod p \end{aligned}$$

be the mod p reduction map. If $P \in E(\mathbb{Q})$ has finite order, and if $\rho_p(P) = \mathcal{O}$, then $P = \mathcal{O}$.

Proof. Since X, Y, Z in $\mathbb{Z}$ may be chosen up to equivalence such that $\gcd(X, Y, Z) = 1$, therefore at least one of $X, Y, Z \bmod p$ is nonzero. Therefore ρ_p is indeed a map from $\mathbb{P}^2(\mathbb{Q}) \rightarrow \mathbb{P}^2(\mathbb{F}_p)$. It is well-defined since the reduction of each coordinate is well-defined. But by theorem 7, if the non-identity affine point $(x, y) \in E(\mathbb{Q})_{tors}$, then $x, y \in \mathbb{Z}$. Thus in projective coordinates we may fix $Z = 1$ for a point of the domain of ρ_p . This means that $\rho_p(X : Y : Z)$ is not the identity in $\mathbb{P}^2(\mathbb{F}_p)$ since $Z = 1 \not\equiv 0 \bmod p$. In other words, if $\rho_p(P) = \mathcal{O}$ then we must have that $P = \mathcal{O}$. □

Remark 12. Actually, $\rho_p|_{E(\mathbb{Q})}$ is a group homomorphism (again, see [Sil09], section VII.3, Proposition 3.1 for a wider context). Theorem 10 shows that the kernel of ρ_p restricted to $E(\mathbb{Q})_{tors}$ is trivial, and therefore ρ_p restricted to $E(\mathbb{Q})_{tors}$ is injective. Thus we say $E(\mathbb{Q})_{tors}$ ‘injects’ into $E(\mathbb{F}_p)$.

Remark 13. From here on we note all that is required is the existence of an injection from $E(\mathbb{Q})_{tors} \rightarrow E(\mathbb{F}_p)$, and since we have shown that such a map exists for a curve in Weierstrass form, it follows that some injection (not necessarily the same map) exists for curves in other forms (e.g. Edwards, Hessian etc). Ultimately we are concerned with divisors of the group order over a finite field, and this is independent of the curve shape.

Remark 14. If the torsion subgroup of E is defined over an extension K of $\mathbb{Q}$ of degree ≥ 2 , then the irrationalities which define K must necessarily exist in $\mathbb{F}_p$ for the primes of interest, if $E(K)_{tors}$ is to inject into $E(\mathbb{F}_p)$. For example, if $E(K)_{tors}$ is defined only for $K = \mathbb{Q}(\sqrt{-1})$, then $\sqrt{-1}$ must necessarily exist in $\mathbb{F}_p$ in order for $E(K)_{tors}$ to inject into $E(\mathbb{F}_p)$. In that case it would be sufficient for p to be a sum of two squares, i.e. $p = a^2 + b^2$, since then $a/b \equiv \sqrt{-1} \bmod p$. Thus there may be certain restrictions on the primes which allow the torsion improvement to work, for the larger torsions possible over higher extensions of $\mathbb{Q}$. In fact, it only works for primes p such that $\mathbb{F}_p$ is the residue field of some prime ideal in $\mathcal{O}_K$, the ring of integers of K .

3.3 Edwards Curves

It may seem surprising that an entire alternative formulation of elliptic curves lay undiscovered for more than a century but this is exactly what happened. In a 2007 paper [Edw07], Harold M. Edwards proved that all elliptic curves over an algebraically closed field K can be written in ‘Edwards normal form’

$$x^2 + y^2 = c^2(1 + x^2y^2). \quad (3.1)$$

This led to a series of investigations by Bernstein, Lange, et al, who generalised curves in Edwards normal form to

$$x^2 + y^2 = 1 + dx^2y^2. \quad (3.2)$$

These curves became known as Edwards curves. This was further generalised in [BBJ⁺08] to the form

$$ax^2 + y^2 = 1 + dx^2y^2. \quad (3.3)$$

Curves in this form are known as twisted Edwards curves. A key point is that not all elliptic curves can be written in Edwards normal form if the field K is not algebraically closed, for example if K is a finite field. Thus we cannot take any elliptic curve and expect it to be representable in the form 3.1 over a finite field. The generalised forms make it possible to represent a higher proportion of isomorphism classes of elliptic curves over any finite field. For example, in [BL07], section 2, it is shown why more than 1/4 of all isomorphism classes of elliptic curves over a finite field are representable in the form 3.2. Formulas for the exact number of isomorphism classes of Edwards and twisted Edwards curves over finite fields are given in [FMW12].

One of the striking features of the various Edwards formulations is the simplicity/symmetry of the group law. The group law for Edwards curves is

$$(x_1, y_1) \oplus (x_2, y_2) = \left(\frac{x_1y_2 + x_2y_1}{1 + dx_1y_1x_2y_2}, \frac{y_1y_2 - x_1x_2}{1 - dx_1y_1x_2y_2} \right) \quad (3.4)$$

The group law for twisted Edwards curves is very similar.

$$(x_1, y_1) \oplus (x_2, y_2) = \left(\frac{x_1y_2 + x_2y_1}{1 + dx_1y_1x_2y_2}, \frac{y_1y_2 - ax_1x_2}{1 - dx_1y_1x_2y_2} \right) \quad (3.5)$$

These formulas are also strongly unified, i.e. they can be used both to add and double a point, something that is not possible, for example, with the Weierstrass addition formulas. Also, provided d is not a square in the field of definition K , the formulas 3.4 and 3.5 are complete - i.e. they can be used to add any two input points defined over K . If d is a square in K , there may be exceptional points for which the formulas are not guaranteed to give correct output. See [BBJ⁺08] for proof of completeness when d is not a square, and theorem 1 of [HWCD11] for a condition for completeness.

It is possible to clear the denominators of 3.4 and 3.5 to produce formulas for the group

law in projective coordinates. Another approach, by Hüseyin Hisil in [His10], is to systematically search for the formulas themselves algebraically. This produced previously unknown formulas, in some cases more efficient for certain curve types.

However the formulas for addition/doubling are derived, these are what determine the speed of scalar multiplication. The formulas are typically given in terms of the number of ring/field multiplications (M), ring/field squarings (S), multiplications by a curve parameter (a and d), additions/subtractions (add) and sometimes inversions (I). For example (see [BL]), the best known formula for twisted Edwards doubling in projective coordinates, assuming field/ring multiplications cost the same as squarings, is

$$\textit{Calculate} : \quad [2](X_1 : Y_1 : Z_1) = (X_3 : Y_3 : Z_3)$$

$$\textit{Cost} : \quad 3M + 4S + 1 \cdot a + 6add + 1 \cdot 2$$

$$B = (X_1 + Y_1)^2$$

$$C = X_1^2$$

$$D = Y_1^2$$

$$E = a \cdot C$$

$$F = E + D$$

$$H = Z_1^2$$

$$J = F - 2 \cdot H$$

$$X_3 = (B - C - D) \cdot J$$

$$Y_3 = F \cdot (E - D)$$

$$Z_3 = F \cdot J$$

We can accomplish twisted Edwards addition in projective coordinates via

$$\begin{aligned}
\text{Calculate : } & (X_1 : Y_1 : Z_1) + (X_2 : Y_2 : Z_2) = (X_3 : Y_3 : Z_3) \\
\text{Cost : } & 10M + 1S + 1 \cdot a + 1 \cdot d + 7add \\
& A = Z_1 \cdot Z_2 \\
& B = A^2 \\
& C = X_1 \cdot X_2 \\
& D = Y_1 \cdot Y_2 \\
& E = d \cdot C \cdot D \\
& F = B - E \\
& G = B + E \\
& X_3 = A \cdot F \cdot ((X_1 + Y_1) \cdot (X_2 + Y_2) - C - D) \\
& Y_3 = A \cdot G \cdot (D - a \cdot C) \\
& Z_3 = F \cdot G
\end{aligned}$$

These are the formulas used in the implementation for twisted Edwards scalar multiplication.

3.4 Hessian Curves

Some elliptic curves can be transformed into Hessian form

$$x^3 + y^3 + 1 = dxy$$

or, in projective coordinates,

$$X^3 + Y^3 + Z^3 = dXYZ \quad (3.6)$$

with a generalization to twisted Hessian form

$$aX^3 + Y^3 + Z^3 = dXYZ \quad (3.7)$$

covering more isomorphism classes of elliptic curves over finite fields. In all cases, it is assumed that $3|d$, here we have absorbed that factor into the constant. We have the following theorem [FJ10] for precisely when conversion to twisted ('generalised' in [FJ10]) Hessian form is possible.

Theorem 11. *Let E be an elliptic curve over a field K . If the group $E(K)$ has a point of order 3 then E is isomorphic over $\overline{K}$ to a twisted Hessian curve. Moreover, if K has an element ω with $\omega^2 + \omega + 1 = 0$, then the group $E(K)$ has a point of order 3 if and only if E is isomorphic over K to a twisted Hessian curve.*

Proof. See [FJ10], theorem 1. □

The formulas for adding two affine points (x_1, y_1) and (x_2, y_2) are

$$(x_1, y_1) \oplus (x_2, y_2) = \left(\frac{x_1 - y_1^2 x_2 y_2}{ax_1 y_1 x_2^2 - y_2}, \frac{y_1 y_2^2 - ax_1^2 x_2}{ax_1 y_1 x_2^2 - y_2} \right) \quad (3.8)$$

The formula for doubling an affine point (x_1, y_1) is

$$[2](x_1, y_1) = \left(\frac{x_1 - y_1^3 x_1}{ay_1 x_1^3 - y_1}, \frac{y_1^3 - ax_1^3}{ay_1 x_1^3 - y_1} \right) \quad (3.9)$$

The (affine) identity element is $(0, -1)$ (in projective coordinates $(0 : -1 : 1)$). The negative of an affine point (x_1, y_1) is $\left(\frac{x_1}{y_1}, \frac{1}{y_1}\right)$. Incidentally, see section 3.5 for a consideration that arises from Hessian negation.

Remark 15. Note that the Hessian (and twisted Hessian) group laws are not necessarily complete (i.e. give correct output for all possible input pairs). In [FJ10], section 4, certain addition laws for $E_{H,a,d}$ over a field K are shown to be complete if and only if a is not a cube in K . The proof of this, proposition 1 in [FJ10], gives a reason that this is not a problem for us in ECM - on the contrary in fact. In the proof, it is shown that if the addition law does not work for points $P_1 = (X_1 : Y_1 : Z_1)$ and $P_2 = (X_2 : Y_2 : Z_2)$ on $E_{H,a,d}$, then the output point $= (X_3 : Y_3 : Z_3)$ will have $X_3 = Y_3 = Z_3 = 0$. This is equal to zero in the field, which in the case of ECM is $0 \bmod p$ for a prime divisor p of the number n we are trying to factor. But this is exactly the breakdown case that we are looking for. Since the computations are done $\bmod n$, if the addition law breaks down in one of the subfields of $\mathbb{Z}/n\mathbb{Z}$, the scalar multiple coordinates will ‘carry along’ the multiple of p and allow it to be recovered in the eventual gcd step that would have been done anyway.

This is just as well, since all the a ’s of the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curves generated turned out to be cubes in $\mathbb{Q}$, and hence in any finite field.

3.5 A Note on the Function ψ

Now that we have described Hessian curves, we expand on a consideration that comes up in practice when using twisted Hessian curves in stage 2. Recall the function ψ of section 2.2.3, which is designed to be the same for a point as for its negative. Since all calculations are done in projective coordinates in order to avoid inversions $\bmod N$, and since the function ψ maps to a rational value (which may be reduced $\bmod N$), we must dehomogenize the coordinate involved in order to ensure

$$\psi(P) = \psi(-P)$$

We must remember that projective coordinates are defined up to equivalence (multiplication by any nonzero scalar multiple). Thus we cannot expect, say, the actual value

of the X coordinate of a point P to be the same as that of the X coordinate of $-P$ in general. This does not pose a problem in the computation of $\psi(P)$ as we can dehomogenize the projective coordinates by dividing through by, for example in the case of Weierstrass curves, the projective Z coordinate (technically we must not do this if $Z \equiv 0 \pmod{p}$) but this is unlikely enough to happen that in practice we ignore this possibility. Note that this is not the same as ignoring the condition we *are* looking for, i.e. that $\psi(P) - \psi(-P) \equiv 0 \pmod{p}$). Thus, we can be sure (in the Weierstrass case) that if $P = (X_1 : Y_1 : Z_1)$ and $-P = (X_2 : Y_2 : Z_2)$, then

$$\frac{X_1}{Z_1} = \frac{X_2}{Z_2}$$

(Note that this holds over the rationals and in $\mathbb{F}_p$). The situation in the case of curves in Edwards form is similar. In that case, the negative of an affine point $P = (x, y)$ is $-P = (-x, y)$, allowing us to use $\psi(P) = y$. If we need projective coordinates, for an Edwards curve E defined over $\mathbb{F}_p$, with a point $P = (X : Y : Z)$, we can still ensure $\psi(P) = \psi(-P)$ by defining $\psi(P) = Y/Z \pmod{p}$ (or in the case of ECM we would need and use $\psi(P) = Y/Z \pmod{N}$).

The situation when using Hessian curves is a bit more complicated. The negative of a point $P = (X : Y : Z)$ in projective coordinates is $(X : Z : Y)$, i.e. the Y and Z coordinates are swapped. This creates a problem since it is not totally obvious how to dehomogenize a point P on a Hessian curve given in projective coordinates, while preserving the requirement that $\psi(P) = \psi(-P)$. We start out in the same way, requiring $\psi(P) = \psi(-P)$, i.e. if $P = (X : Y : Z)$

$$\psi((X : Y : Z)) \equiv \psi((X : Z : Y)) \pmod{p}$$

but we cannot take

$$\psi((X : Y : Z)) \equiv \frac{X}{Z} \pmod{p}$$

(since then $\psi(P) \neq \psi(-P) \equiv X/Y$), and neither can we take

$$\psi((X : Y : Z)) \equiv \frac{X}{Y} \pmod{p}$$

(since then $\psi(P) \neq \psi(-P) \equiv X/Z$) - so the situation does not work exactly as it does for the Weierstrass and Edwards cases, where we are able to dehomogenize both P and $-P$ using a single value in the coordinates (Z in those cases). However, there is a simple solution. We use

$$\psi((X : Y : Z)) \equiv \frac{X^2}{YZ} \pmod{N}$$

and this function does agree with the requirement. This was derived and used in the implementation, only after discovering that factoring in stage 2 using Hessian curves was working about half of the time it should have (mistakenly having originally taken $\psi((X : Y : Z)) = X/Z \pmod{N}$).

3.6 The Jeon-Kim-Lee Families

Part of the unfolding story of classifying possible torsion subgroups for elliptic curves over number fields has been identifying which torsion subgroups can arise infinitely often in non-isomorphic curves. For number fields of degree d , this has been done for $d = 1$ (Kubert, [Kub76]), $d = 2$ (Rabarison, [Rab10]), $d = 3$ (Jeon, Kim, Lee, [JKL11a]) and $d = 4$ (see below). It is still an open question, for example, which torsion subgroups can arise infinitely often for $d = 5$. Of the known cases, we pay particular attention to the case of quartic number fields, since they admit larger torsion subgroups (larger is better for ECM, see section 3.2.3). Explicit parameterisations of curves in torsion families for the quartic case were given in a 2011 paper of Daeyeol Jeon, Chang Heon Kim, and Yoonjin Lee [JKL11b].

3.6.1 Torsion Family $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$

Here we state a theorem of [JKL11b] which allows the generation of curves with $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ torsion over the given quartic number field.

Theorem 12. *Let $K = \mathbb{Q}(\sqrt{-1}, \sqrt{t^4 - 6t^2 + 1})$, with $t \in \mathbb{Q}$ and $t \neq 0, \pm 1$, and let E be an elliptic curve defined by the equation*

$$E : y^2 + xy - \left(\nu^2 - \frac{1}{16}\right)y = x^3 - \left(\nu^2 - \frac{1}{16}\right)x^2$$

where

$$\nu = \frac{t^4 - 6t^2 + 1}{4(t^2 + 1)^2}.$$

Then the torsion subgroup of E over K is equal to $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ for almost all t .

Proof. See [JKL11b], section 4.8. □

In addition, here we describe how to convert a curve in this family to twisted Edwards form. Note that a necessary and sufficient condition for an elliptic curve to be representable in Edwards form is that it has a point of order 4. This is prescribed by any curve in the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family above - since the torsion subgroup is isomorphic to $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$, which contains the point $(1, 2)$ of order 4 (among others). Any Edwards curve with rational parameter d can be converted into an isomorphic twisted Edwards curve with integer parameters.

Once we have the parameters d and a of the corresponding Edwards curve, we can then construct the isomorphic curve

$$E : y^2 = x^3 + 2(a + d)x^2 + (a - d)^2x$$

Algorithm 5 Generate Edwards Curve

Input:

Elliptic curve in Weierstrass form with a point of order 4

Output:

Parameters d, a of an isomorphic twisted Edwards curve $ax^2 + y^2 = 1 + dx^2y^2$

```
1: procedure EDWARDSGEN
2:    $E \leftarrow$  elliptic curve in Weierstrass form with a point of order 4
3:    $P = (x_1, y_1) \leftarrow$  point of order 4 on  $E$ 
4:   Suppose  $E$  has the form  $E : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$ 
5:   if  $a_1 \neq 0$  or  $a_3 \neq 0$  then
6:      $E \leftarrow$  substitute  $y = y - (a_1x + a_3)/2$ 
7:      $P \leftarrow (x_1, y_1 + (a_1x + a_3)/2)$ 
8:   end if
9:    $P_2 \leftarrow [2]P = (x_2, y_2)$ 
10:  if  $P_2 \neq (0, 0)$  then
11:     $E \leftarrow$  substitute  $x = x + x_2$ 
12:     $P \leftarrow (x_1 - x_2, y_1)$ 
13:  end if
14:   $(r, s) \leftarrow P$ , where  $r, s \in \mathbb{Q}$ 
15:   $d/a \leftarrow 1 - 4r^3/s^2$ , where  $d \in \mathbb{Z} \setminus \{0, 1\}, a \in \mathbb{Z} \setminus \{0\}$  are coprime
16:  This defines a twisted Edwards curve  $E_{a,d} : ax^2 + y^2 = 1 + dx^2y^2$  such that  $E_{a,d} \cong E$ 
17: end procedure
```

and find a point (u, v) on E . To find a corresponding point (x, y) on $E_{a,d}$, compute

$$x = \frac{u}{v}$$
$$y = \frac{u - a + d}{u + a - d}$$

We can take the absolute values of x and y .

3.6.2 Torsion Family $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$

Here we state a theorem of [JKL11b] which allows the generation of curves with $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ torsion over the given quartic number field.

Theorem 13. *Let $K = \mathbb{Q}(\sqrt{-3}, \sqrt{8t^4 + 1})$, with $t \in \mathbb{Q}$ and $t \neq 0, 1, -\frac{1}{2}$, and let E be an elliptic curve defined by the equation*

$$E : y^2 = x^3 - 27\mu(\mu^3 + 8)x + 54(\mu^6 - 20\mu^3 - 8)$$

where

$$\mu = \frac{2t^3 + 1}{3t^2}.$$

Then the torsion subgroup of E over K is equal to $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ for almost all t .

Proof. See [JKL11b], section 4.9.

□

We can easily write down the equation for these curves in twisted Hessian form. The parameter μ above is actually already the parameter of the Hessian curve

$$X^3 + Y^3 + Z^3 = 3\mu XYZ$$

We let $d/a = 3\mu$, where $d \in \mathbb{Z} \setminus \{0, 1\}, a \in \mathbb{Z} \setminus \{0\}$ are coprime. To get a twisted Hessian curve we simply ‘twist’ the Hessian curve having parameter d by a to get

$$E_H : aX^3 + Y^3 + Z^3 = dXYZ$$

To find a point (x, y) on E_H from an affine point (u, v) on E we compute

$$\begin{aligned} x &= \frac{36(\mu^3 - 1) - v}{6(u + 9\mu^2)} - \mu/2 \\ y &= \frac{v + 36(\mu^3 - 1)}{6(u + 9\mu^2)} - \mu/2 \end{aligned}$$

3.6.3 A Coincidence

Bernstein, Lange et al in [BBLP13] provided a new implementation of ECM, named EECM-MPFQ because it uses Edwards curves and the MPFQ library for large integer arithmetic. As well as a speedup due to the faster group law formulas of Edwards and twisted Edwards curves, another speedup came from the use of curves with small parameters and point coordinates (see section 2.1.6 for how this gives a speedup). To find such curves, an (enhanced) exhaustive search was carried out, taking about a week of computation time on a number of computers. The searches yielded 78 curves with torsion $\mathbb{Z}/12\mathbb{Z}$, and 25 curves with torsion $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$. The list of found curves is published at <http://eecm.cr.yp.to/goodcurves.html>. It turns out that the curves produced by the Jeon-Kim-Lee $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ and $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ families, by generating curves via the rational parameter t increasing in height, end up with small parameters, including when converted to Edwards/Hessian form. Not only this, but searching for a point of infinite order for curves with positive rank always seemed to yield a point with small projective coordinates. This perfectly suits an implementation of ECM needing the small parameter speedup.

However, there were a couple of unexpected coincidences. It was originally hoped that the Jeon-Kim-Lee families would outperform the EECM-MPFQ ‘good curves’ referred to above, since those curves are listed in terms of their torsion over $\mathbb{Q}$, and the Jeon-Kim-Lee quartic number field torsion families achieve torsions not possible over $\mathbb{Q}$. But examining the output of curves generated from the Jeon-Kim-Lee $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family revealed all 25 of the $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves. After some experimentation in an attempt to find the $\mathbb{Z}/12\mathbb{Z}$ curves, it was found that many of them appear in the Jeon-Kim-Lee $\mathbb{Z}/24\mathbb{Z}$ family (not used in the implementation, since it is effectively superseded in performance by the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family). So the EECM-MPFQ curves really have higher torsion over a quartic number field, which in fact is explicitly given in the definition of the Jeon-Kim-Lee families.

	# Curves	Type	Torsion	Time	Resources
EECM-MPFQ	25	Edwards	$\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$	1 week	Several computers
EECM-MPFQ	75	Edwards	$\mathbb{Z}/12\mathbb{Z}$	1 week	Several computers
This Thesis	700	Edwards	$\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$	2 weeks	One desktop computer
This Thesis	4840	Hessian	$\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$	2 weeks	One desktop computer

Table 1: Resources for generation of small-parameter curves for ECM

This is good news and bad news. On the bad side, it means we can't automatically outperform the EECM-MPFQ curves by making use of higher torsion orders over a quartic number field - our curves are the very same curves.

On the other hand, on the good side, we can generate curves *much* more quickly than was needed for the EECM-MPFQ curves - obviously since we have a parameterised family. It was an unexpected bonus that both the curve parameters and point of infinite order coordinates were small. This allowed the generation of hundreds of $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves, and in fact thousands of $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curves, with unique j-invariants (sometimes j-invariants were duplicated in each family but there remained plenty of unique curves). So overall, we win - the drawback of EECM-MPFQ is that it only had around 100 high-performance curves in total to draw on for any factoring attempt - this would effectively stop it from finding record-sized primes, where usually it is necessary to try thousands of curves on each number (however this may not have been the intended application of EECM-MPFQ, which may have been geared more towards use as a factoring subroutine to speed up the number field sieve).

Finally, for comparison, we illustrate the resources required to find high-performance curves for ECM, for this work and EECM-MPFQ (see table 1).

3.6.4 Visualization of Curves with Positive Rank

The curves in the Jeon-Kim-Lee $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ are parameterised by a single rational number t . Not every t produces a curve of positive rank. It is informative to plot the values of t in this family that do give a curve of positive rank. This is done below in figure 1, for 700 data points, where the numerator is plotted against the denominator. Note that since the formula for ν , which determines the curve, is

$$\nu = \frac{t^4 - 6t^2 + 1}{4(t^2 + 1)^2}.$$

the plot is symmetric about the vertical axis ($-t$ gives the same curve as t). Also, since substituting $1/t$ for t in the formula for ν results in the same value of ν , the plot is also symmetric about the diagonal.

The curves in the Jeon-Kim-Lee $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ are parameterised by a single rational

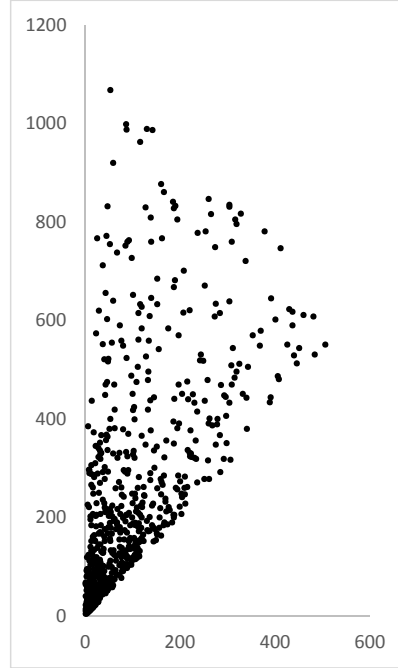


Figure 1: Numerator vs denominator for t producing positive rank $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves

number t . Not every t produces a curve of positive rank. The numerator vs denominator of t giving a curve of positive rank is plotted in figure 2, for a radius of about 300 (i.e. the distance to the origin of numerator vs denominator is less than about 300). There are a few thousand data points. Note that this curve is not symmetric about the vertical axis (nor the diagonal).



Figure 2: Numerator vs denominator for t producing positive rank $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curves

3.6.5 Other Torsion Families

At least one other torsion family was considered (in [JKL11b] many families are described). Of note is the $\mathbb{Z}/24\mathbb{Z}$ family, from which curves may be converted to Edwards form, since the torsion subgroup contains a point of order 4 (since $6 \in \mathbb{Z}24$ and $6 \cdot 4 \equiv 24 \equiv 0 \pmod{24}$). However, there was already a family allowing conversion to Edwards form, with even higher torsion, namely the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family. Also, if it is nice to have a factor 3 in the group order, this is supplied by the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family, which also has larger torsion. For these reasons, the $\mathbb{Z}/24\mathbb{Z}$ family was not used in the implementation, although it might have been a good source of extra curves.

Not much attention was given to the other families in [JKL11b]. Some of these families are potentially useful for numbers of a certain form if the elliptic curves generated can be converted into a form with fast arithmetic. Although impossible to convert to Edwards or twisted Edwards form (or even to find an isogenous twisted Edwards curve since there are not 3 points of order 2) and also impossible to convert to Hessian or twisted Hessian form (not having a point of order 3 by theorem 11 precludes this) the $\mathbb{Z}/5\mathbb{Z} \oplus \mathbb{Z}/5\mathbb{Z}$ family can be used, as pointed out by Brier and Clavier in a slightly different context (requiring infinitely many curves of positive rank - here we still have to search for such curves) in [BC10], to factor numbers of the form $a^{5^n} \pm 1$ with a guaranteed group order divisor of 25.

Nothing has been said about finding curves with torsion subgroups only possible over number fields of degree higher than 4 - but see section 3.9 for aspects of this kind of search.

3.7 Alternative - The Atkin-Morain Family

A total of 700 curves from the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family were found for the implementation (see section 4.2). It becomes progressively harder to find new positive rank curves (at least heuristically from the experimental data) as the distance from the origin of the rational parameter t increases. It may not be feasible to accumulate (many) thousands of these curves. Although positive rank curve generation is faster, the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family has one drawback in particular - for general prime fields, the guaranteed divisor is not as large as is possible for a family over $\mathbb{Q}$. The $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves (although this was not proved) seem to have $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ torsion over $\mathbb{Q}$ (resulting in the maximum prescribed group divisor of 16 possible for a curve defined over $\mathbb{Q}$), whereas the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curves require $\sqrt{-3}$ to exist in the prime field to achieve the injection of $\mathbb{Z}/3\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ torsion (resulting in a prescribed group divisor of 18), and otherwise they have torsion $\mathbb{Z}/6\mathbb{Z}$ over $\mathbb{Q}$. Both families require the existence, with roughly a 50% chance, of yet another quadratic irrationality in the prime field to achieve full torsion injection. Alternatively, in e.g. [BBLP13], Section 7.1, we have the Atkin-Morain parameterised family with torsion subgroup isomorphic to $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$, with an explicit non-torsion point given. This not only allows the generation of infinitely many curves with maximum possible torsion over $\mathbb{Q}$ (and hence a guaranteed group order divisor of 16 for any prime of good reduction),

but also provides the point of infinite order we need for ECM.

Theorem 14 Atkin-Morain, adapted to Edwards form in [BBLP13]. Let (s, t) be a rational point on the curve

$$T^2 = S^3 - 8S - 32$$

(e.g. multiples of $(12, 40)$). Define $\alpha = 1/((t + 25)/(s - 9) + 1)$, $\beta = 2\alpha(4\alpha + 1)/(8\alpha^2 - 1)$, and

$$d = \frac{(2(2\beta - 1)^2 - 1)}{(2\beta - 1)^4}.$$

Then the Edwards curve

$$E_d : x^2 + y^2 = 1 + dx^2y^2$$

has torsion group isomorphic to $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ and a point (x_1, y_1) with

$$x_1 = \frac{(2\beta - 1)(4\beta - 3)}{(6\beta - 5)}$$

and

$$y_1 = \frac{(2\beta - 1)(t^2 + 50t - 2s^3 + 27s^2 - 104)}{(t + 3s - 2)(t + s + 16)}$$

which has infinite order over $\mathbb{Q}$.

Proof. See [BBLP13], Theorem 7.3. □

Remark 16. Curves generated by this construction have rapidly increasing height over $\mathbb{Q}$, but this does not mean they cannot be used in ECM. Rather than compute the point (x_1, y_1) over $\mathbb{Q}$, we compute the point in $\mathbb{Z}/n\mathbb{Z}$ (which the construction allows). We do the same for d . This gives us an Edwards curve and point defined over $\mathbb{Z}/n\mathbb{Z}$ which we may then use for ECM. The only drawback is that we will basically lose the ‘small parameters’ advantage, as the curve parameters and point coordinates will likely be of the size of n . But if we lose small parameters, we regain the ability to use Montgomery reduction (see section 2.1.4), while keeping the fast arithmetic of Edwards and twisted Edwards curves. This approach was not implemented but it could be a useful investigation.

3.8 Hessian Curve Advantage in Scalar Multiplication

If we have a twisted Hessian curve with small parameters and a fixed point of infinite order with small projective coordinates, there is a substantial saving that can be made. Note that in this thesis, over 4,800 such curves were found. We can effectively ‘reduce’ the operation count of a twisted Hessian curve addition, in the specific context of scalar multiplication of a fixed point, from $12M + 1a + 3add$ to something approaching $6M + 3add$. Of course, to actually meet the latter operation count, all point coordinates and curve parameters would have to be equal to 1, but nevertheless there is a significant speedup to be made, and in fact it even allows Hessian curves to compete with Edwards curves in terms

of performance of scalar multiplication. To qualify that statement, note that this only applies to twisted Hessian addition - not twisted Hessian doubling. Since doublings cannot really take advantage of small coordinates/parameters, the cost of doubling remains the same. However, what has happened is that the impact of additions is considerably reduced.

The result of certain modular multiplications now is not much larger than N . This means that not only are some of the modular multiplications much faster, but we can also delay modular reductions - no longer do we need to reduce the result of a multiplication mod N immediately. The form of the elliptic curve addition formula facilitates this delay - there are a number of integer multiplications that can be ‘carried along’ because they do not result in a number much bigger than N , with a final reduction (that would have been necessary anyway) on each of the projective X , Y and Z values of the output point. In effect, it is as though we have rendered windowing techniques to minimize the number of additions at least partially unnecessary. Windowing has increasing memory/precomputational costs, so we have offset these significantly because small parameters reduce the overall cost of additions.

The twisted Hessian curve addition law (one of a number of possibilities) is reproduced below so that the idea of this speedup is clear. Note that $(X_1 : Y_1 : Z_1)$ is fixed, with small coordinate values, and likewise the curve parameter a is small (d is typically small too but it is not used in point addition).

Remark 17. Note that not using windowing avoids certain issues, such as having to contend with a higher memory cost for stage 1, and corresponding precomputation, as well as avoiding extra program complexity.

Input: Points $(X_1 : Y_1 : Z_1), (X_2 : Y_2 : Z_2)$ on a twisted Hessian curve E

with equation $aX^3 + Y^3 + Z^3 = dXYZ$

Output: Point $(X_3 : Y_3 : Z_3) = (X_1 : Y_1 : Z_1) + (X_2 : Y_2 : Z_2)$ on E

$$A = X_1 \cdot Z_2$$

$$B = Z_1 \cdot Z_2$$

$$C = Y_1 \cdot X_2$$

$$D = Y_1 \cdot Y_2$$

$$E = Z_1 \cdot Y_2$$

$$F = a \cdot X_1 \cdot X_2$$

$$X_3 = A \cdot B - C \cdot D$$

$$Y_3 = D \cdot E - F \cdot A$$

$$Z_3 = F \cdot C - B \cdot E$$

3.9 Even Larger Torsion

We could ask the question, why stop at torsion over quartic number fields - why not keep going and search for families of curves giving even higher guaranteed divisors of the group order over a finite field?

Note that an inherent part of a search such as this is finding the splitting field of a division polynomial of a curve E .

Definition 3. *If $\text{char}(K) \neq 2, 3$, given an elliptic curve over K in short Weierstrass form*

$$E : y^2 = x^3 + Ax + B$$

the division polynomials Ψ_n are elements of the ring $K[x, y]/\langle y^2 - x^3 - Ax - B \rangle$ (the ones which we will use belong to the subring $K[x]$) defined for each $n \in \mathbb{N}$ by the following recursion:

$$\Psi_0(x, y) = 0$$

$$\Psi_1(x, y) = 1$$

$$\Psi_2(x, y) = 2y$$

$$\Psi_3(x, y) = 3x^4 + 6Ax^2 + 12Bx - A^2$$

$$\Psi_4(x, y) = 4y(x^6 + 5Ax^4 + 20Bx^3 - 5A^2x^2 - 4ABx - A^3 - 8B^2)$$

$$\Psi_{2m+1}(x, y) = \Psi_{m+2}(x, y)\Psi_m^3(x, y) - \Psi_{m-1}(x, y)\Psi_{m+1}^3(x, y) \quad \text{for } m \geq 2$$

$$\Psi_{2m}(x, y) = \frac{\Psi(x, y)}{\Psi_2(x, y)} (\Psi_{m+2}(x, y)\Psi_{m-1}^2(x, y) - \Psi_{m-2}(x, y)\Psi_{m+1}^2(x, y)) \quad \text{for } m \geq 3$$

By theorem 10 of section 3.2.3, E must have good reduction modulo the prime field $\mathbb{F}_p$ of interest. If the required (large) torsion group only exists over a field K which is an extension of $\mathbb{Q}$, then a necessary condition for the torsion to inject into $E(\mathbb{F}_p)$ is that the minimal polynomial of the primitive element of K must split fully over $\mathbb{F}_p$. Here is an example (where the computation was done using SAGE). Let E be the Edwards curve with equation

$$ax^2 + y^2 = 1 + dx^2y^2$$

where $(d, a) = (25921, 83521)$ (this is actually the first curve in the list of Edwards curves used in the implementation). Let us consider the 8th division polynomial ψ_8 for E . Whereas ψ_8 itself is of degree 33 and has some large coefficients, the minimal polynomial $f(x)$ of the primitive element of the splitting field K of ψ_8 can be written down easily, and is equal to

$$f(x) = x^{16} + 156x^{14} + 34806x^{12} + 1064472x^{10} + 142609417x^8 - 14516581188x^6 + 445832781396x^4 - 6153609718464x^2 + 32752545864256$$

The discriminant Δ of E is

$$\Delta = 6100667232581834086809600000000 = 2^{40}3^85^87^217^423^2$$

It so happens (as verified using SAGE) that E/K has torsion subgroup isomorphic to $\mathbb{Z}/8\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$, rather larger than we are perhaps used to seeing. So this elliptic curve will have a head start in trying to find any prime p that does not divide Δ , and such that $f(x)$ splits completely over $\mathbb{F}_p$, since $\#E(\mathbb{F}_p)$ will necessarily be divisible by 64.

Here is another example. Let E be the Edwards curve with equation

$$ax^2 + y^2 = 1 + dx^2y^2$$

where $(d, a) = (1936000000, 1932101119)$ (this is a curve from the Jeon-Kim-Lee $\mathbb{Z}/24\mathbb{Z}$ family, basically not used since it is outperformed by $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family. The $\mathbb{Z}/24\mathbb{Z}$ curves admit conversion to Edwards form since they have a point of order 4). Let us consider the 6th division polynomial ψ_6 for E . The minimal polynomial $f(x)$ of the primitive element of the splitting field K of ψ_6 is equal to

$$\begin{aligned} f(x) = & x^{12} - 6x^{11} + 118694x^{10} - 474738x^9 + 9396396621x^8 + 1414671745896x^7 \\ & + 434012304885770x^6 + 27786422890853286x^5 + 12714678704928922849x^4 \\ & - 3398473479954151103070x^3 + 483497317418162112774786x^2 \\ & - 47059118945065049909426172x + 4238755461992496987735300324 \end{aligned}$$

The discriminant Δ of E is

$$\Delta = 2^{18}3^{12}5^67^{12}11^213^317^3179421^4$$

This curve E/K has even larger torsion, isomorphic to $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/12\mathbb{Z}$, with a guaranteed factor of 72 in $\#E(\mathbb{F}_p)$ for primes p of good reduction, where $f(x)$ splits fully.

Remark 18. Note that these are not just isolated examples. This approach should work for every curve that can be generated from the corresponding family. However, here is the catch - the applicability of any curve in particular becomes much more selective. For the curves with larger torsion over quartic number fields, the configuration of the quartic (biquadratic) number field was very simple - we required that -1 (for the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family, or -3 for the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family) have a square root in $\mathbb{F}_p$, and that one other element have a square root in $\mathbb{F}_p$. This is not too unlikely, as half of the nonzero elements in an odd prime field are quadratic residues, and in fact often we can detect in advance whether certain elements exist in the finite field - for example $\sqrt{-1}$ exists in $\mathbb{F}_p$ for any p dividing a number of the form $A^2 + B^2$ (e.g. Fermat numbers). The number of primes for which the minimal polynomial splits fully becomes proportionately smaller as the degree increases. We could arrange to factor numbers of a very specific form in each case, for example contriving a number by just evaluating $f(x)$ at some x , but this loses

generality and in any case only works on a curve-by-curve basis. However, it might be something worth further investigation to see if something useful can be discovered in this regard. For example, if true, it can probably be easily shown that the torsion subgroup $\mathbb{Z}/8\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ occurs infinitely often in elliptic curves over number fields of degree 16, and torsion $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/12\mathbb{Z}$ occurs infinitely often in number fields of degree 12. See [CCRS14] for a computation which determines the possible torsion subgroups of elliptic curves with ‘complex multiplication’ (a subset of all curves) for number fields up to degree 13. In particular, it is shown that for complex multiplication (CM) curves, over a number field of degree 12, the largest possible torsion subgroup has structure $\mathbb{Z}/7\mathbb{Z} \oplus \mathbb{Z}/7\mathbb{Z}$ (it was not determined if this subgroup occurs infinitely often).

4 The Implementation

4.1 Overview

The primary motivation of the implementation was to see if the torsion/small parameter improvements would lead to an implementation that was competitive with either GMP-ECM or EECM-MPFQ. It was decided at the outset to write a C++ application targeting a 64-bit environment. In practice, the established ECM software when implemented on a 64-bit machine greatly outperforms an implementation on a 32-bit machine. Another consideration is that a 32-bit machine cannot address more than 4Gb of memory more or less by definition, and in taking advantage of multiple cores (ECM is very easily parallelised), it was quite likely that more than 4Gb would be needed. Finally, an end goal of this work was to test an implementation on the Fionn cluster at the Irish Centre for High-End Computing (ICHEC), which uses 64-bit cores.

Fionn (fionn.ichec.ie) is a heterogenous machine made up of four components: Thin, Hybrid, Fat and Service.

The Thin component is an SGI ICE X system of 320 nodes or 7680 cores made of 2.4GHz Intel Ivy Bridge cores. Each node has 2x12 core processors, 64GB of RAM and is connected using FDR InfiniBand. This amounts to 20TB of RAM across the partition.

The bulk of the computation done used the Thin component. The largest allowable batch is a 72 hour run, utilising 1,032 cores simultaneously. This large batch size was run twice. Other batches were smaller, during testing, for example, culminating in the two large batches. The Service component provides login/scripting/other front-end functionality.

As mentioned in section 2.1.3, a necessary part of factoring large integers on a computer is having a fast library to handle big integer arithmetic. GNU-MP (Gnu-Multi Precision/GMP) was chosen as it is well established, being used by GMP-ECM. MPFQ was another contender, however the number of words in integers being factored must be known at compile-time, and a compiled program can therefore only handle a certain number of predetermined sizes. While this may have been a good idea (especially as it appears MPFQ arithmetic is a good deal faster than corresponding GMP arithmetic), it lacked a certain freedom that it was desired that the implementation have. It was desired to be able to feed any (reasonably) large number into the program and expect results, from factoring, say, 30 digit integers, to attempting to find unknown factors of Fermat numbers (the smallest of these with an incomplete factorisation, at the time of writing, being F12, having 1234 digits/4096 bits/64 64-bit words). Another consideration is that GMP implements FFT multiplication of integers automatically past a certain threshold, giving a tangible speed increase in stage 2 as a result, whereas MPFQ does not seem to have this capability. Primarily for these reasons, GMP was preferred over MPFQ.

4.2 SAGE and Generating Curves with Positive Rank

Another aspect of the implementation was not the program itself, but the generation of large numbers of curves from the two Jeon-Kim-Lee torsion families, $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ and $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$, for the program to use. This involves the manipulation of elliptic curves and, for example, verification of torsion properties. This would really have been infeasible without using a package such as MAGMA or SAGE. SAGE was used because it provides the necessary functionality and is free.

Scripts were written to generate curves from the Jeon-Kim-Lee families for as many values of the parameter t as possible, keeping those with positive rank. Although not known beforehand, the density of curves with positive rank appears to be greatest for t near the origin of the space of numerator vs denominator, with the density reducing further out. Rather than try values of t in blocks, it was decided to traverse the rationals in the manner of Cantor's 'diagonal argument' for proving the uncountability of the reals. This is done via algorithm 6. Curves are generated more quickly if dense regions are searched first.

In total, the final list generated from the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family for use by the implementation had 700 curves, and the list from the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family (after removing isomorphic and $\mathbb{Q}$ -isogenous curves, see next section) had 4840 curves. Excerpts from these lists are given in appendices A.4 and A.5.

4.3 A Problem Arising from Isogenies between Curves

Part of verifying correct operation of the implementation led to checking elliptic curve group order factorisations over various finite fields. It was noticed that when $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curves were used, often different curves would produce the same group order factorisations, even for large prime fields. This shouldn't really be allowed to happen, since then work is unnecessarily duplicated. The following theorem explains what was happening.

Theorem 15 *Tate's Isogeny Theorem.* *Two elliptic curves are isogenous over a finite field if and only if they have the same number of points.*

Proof. See [Tat66], p. 139.

□

Since this seemed to be happening between two given curves for many (all tested) primes, we might expect there is something deeper happening. Without giving a rigorous justification, it is likely that there is a 'local to global' principle at work. If so, then if there is an isogeny between E_1 and E_2 over every prime field $\mathbb{F}_p$ (local fields), in fact there is an isogeny between E_1 and E_2 over $\mathbb{Q}$ (a global field). If this is the case, then including two curves between which there is such an isogeny over $\mathbb{Q}$ makes no sense for ECM - there is no benefit in running the second curve.

Algorithm 6 Traverse rationals for parameter t via Cantor's method

Input:

Family of elliptic curves parameterised by $t \in \mathbb{Q}$

Output:

Curves for approximate increasing height of t

```
1: procedure CURVEGEN
2:    $numerator \leftarrow 1$ 
3:    $denominator \leftarrow 1$ 
4:    $direction \leftarrow right$ 
5:   for  $i = 1$  to  $maximum$  do
6:     if  $direction = right$  then
7:       if  $numerator > 1$  then
8:          $numerator = numerator + 1$ 
9:       else
10:         $direction = left$ 
11:      end if
12:       $denominator = denominator + 1$ 
13:    else
14:      if  $denominator > 1$  then
15:         $denominator = denominator - 1$ 
16:      else
17:         $direction = right$ 
18:      end if
19:       $numerator = numerator + 1$ 
20:    end if
21:     $t \leftarrow numerator/denominator$ 
22:    if  $\gcd(numerator, denominator) = 1$  then
23:      Generate curve with parameter  $t$ 
24:    end if
25:  end for
26: end procedure
```

There is an established method, due to Vélú [Vél71], for determining all curves that are isogenous to a given curve, however in the most general case, the formulas become unwieldy. Since a practical way of identifying $\mathbb{Q}$ -isogenous curves was quickly needed at the time, it was sufficient to compute some large-ish prime p , then compute the group order factorisations of $\#E(\mathbb{F}_p)$ for all curves E in the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ list, and cull the curves with larger parameters that had duplicate factorisations. This reduced an initial list of over 8,400 curves to just over 4,800. So it would seem that there are lots of isogenous curves in the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family. In addition, there are a fair amount of isomorphic curves to begin with, which were easier to detect as for each curve generated, its j -invariant was recorded, and any duplicate curves were culled - there were over 9,200 curves generated originally.

4.4 The Development Process

The following (obvious) approach to implementation was used:

1.	Construct outline of code
2.	Verify logic (follow program flow)
3.	Correct syntax
4.	Verify execution (via debugging/examination of output)
5.	Clean up (addressing any remaining issues, such as memory management)

4.4.1 Working with Pari/GP, C++, Java

To aid in what was a substantial undertaking (including implementation of the FFT continuation from scratch) for not much more than a single semester, a proof of concept was typically constructed in PARI/GP before attempting part of the final implementation, to try to prevent major problems (conceptual or otherwise) from occurring at what might have been a critical time.

4.4.2 Memory Leaks and Valgrind

When the implementation was complete and working (i.e. finding factors in the expected way), a residual problem was detected. The memory used by the program was slowly increasing (large memory leaks were fixed early on). An invaluable tool in this regard to fix the remaining problems was Valgrind, which can give line numbers at which memory was allocated, that was not released.

4.4.3 A Note on Assembly Language

Assembly language was not used at all in the implementation due to time constraints. If it had been used, it probably would have allowed the software to set stage 1 speed records.

For example, GMP-ECM makes use of assembly language optimizations that compensate for certain inefficiencies in the GMP library, and yet (see the section Results) stage 1 of this implementation almost achieved the speed of stage 1 of GMP-ECM during certain trials (consistently on the same machine, with the same bounds).

4.5 A Smartphone App

As a novelty, a ‘Google Android’ smartphone app implementing stage 1 of ECM was written and deployed to a Samsung Galaxy S3, with the objective of being the top factoring app on this platform. It was found that the Java library for large integer arithmetic can be prohibitively slow, but extra optimizations written in C++ may be possible to make ‘Fast Factors’ (the name given to the app) the clear winner.

5 Computational Aspects

5.1 Which Numbers?

With the implementation complete, there are many and varied factoring projects providing sources of numbers to test the program with, both numbers with discovered factorization, and (as yet) unknown factorization. This leads to being able to choose many numbers with a given number of digits or within a given digit range for testing. One aspect of this however, is that due to the efforts of the factoring community, many (most?) of the easy-to-factor numbers taken from these projects have already been successfully factored. An aspect of factoring efforts in practice is to be reasonably sure progress is being made as more and more ECM effort is being applied, at progressively higher bounds. There are various recommended bounds, together with expected number of curves to find a factor of n digits, summarised as e.g. ‘t35’, ‘t40’, ‘t50’, where the number is the number of digits and ‘t’ probably stands for ‘target’ or ‘test’. Examples of this are:

t35 - number tested up to 35-digit factors (948 curves with $B1=1e6$)
t40 - number tested up to 40-digit factors (2,440 curves with $B1=3e6$)
t45 - number tested up to 45-digit factors (4,590 curves with $B1=11e6$)
t50 - number tested up to 50-digit factors (7,771 curves with $B1=43e6$)
t55 - number tested up to 55-digit factors (17,899 curves with $B1=110e6$
or 50,000 curves with $B1=43e6$)

(see <http://www.loria.fr/~zimmerma/records/ecm/params.html> for a longer list)

Consequently, where all numbers in a given project have passed ‘t35’ for example, it is very unlikely that there is a number in a project with a 35 digit or less factor, and often much more effort than this has been applied to most/all numbers. This meant in order to have moderate success in finding 40-50+ digit numbers, a source of many unfactored, untested numbers was needed. See section 5.1.2 for the family of numbers that was most often used.

5.1.1 Cunningham Numbers

Cunningham numbers are numbers of the form

$$a^b \pm 1 \quad \text{where } a \in \{2, 3, 5, 6, 7, 10, 11, 12\}$$

The allowed values of b are roughly between 1 and 1000 but the accepted exponents change with the size of the base and the upper limit on size. The Cunningham project is to simply factor all Cunningham numbers, and much work has been put into this. Occasionally the list of remaining numbers is depleted and the range of allowed exponents is increased.

In fact the special structure of some Cunningham numbers allows us to predetermine the existence of some or all of the quadratic irrationalities defining the biquadratic number

field of elliptic curves in certain Jeon-Kim-Lee families, in the finite field of interest in ECM. For example, -3 is a quadratic residue in $\mathbb{F}_p$ (i.e. $\sqrt{-3}$ exists in $\mathbb{F}_p$) for any p dividing numbers of the form

$$1 + 3^{2m+1} = 1 + 3 \cdot 3^{2m} \quad m \in \mathbb{Z}^+$$

and also

$$1 + 12^{2m+1} = 1 + 3 \cdot (3^m 2^{2m+1})^2 \quad m \in \mathbb{Z}^+$$

For the Jeon-Kim-Lee $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family, the associated biquadratic (quartic) number field is $K = \mathbb{Q}(\sqrt{-3}, \sqrt{8t^4 + 1})$, with $t \in \mathbb{Q}$ and $t \neq 0, 1, -\frac{1}{2}$. If -3 is a quadratic residue in $\mathbb{F}_p$, and $8t^4 + 1$ is not, then the torsion is actually $\mathbb{Z}/3\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ (for primes of good reduction). If $8t^4 + 1$ is a quadratic residue as well, then the torsion is $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$. So if -3 at least is a quadratic residue, then we have a guaranteed factor of at least 18 dividing $\#E(\mathbb{F}_p)$ for any p dividing the number, with roughly a 50% chance that 36 divides $\#E(\mathbb{F}_p)$. Both are larger than anything possible over $\mathbb{Q}$. The case of the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family is similar.

5.1.2 Other Numbers

Generalising the form of certain Cunningham numbers, the main source of numbers to test the implementation on were of the form

$$a^{2m} + 3b^{2n} \quad \text{where } a, b, m, n \in \mathbb{Z}^+ \quad (5.1)$$

As far as could be ascertained, there is no current factoring project focussing exclusively on this set of numbers, allowing numbers of a wide range of sizes to be tested (with the possibility of breaking new ground). For this reason, and owing to the higher minimum torsion in the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family, numbers of the form 5.1 were predominantly used in the factoring efforts, as opposed to numbers facilitating torsion in the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family.

5.2 Current Records and Record Attempts

At the time of writing, the record size factor to date found by ECM is the 83 digit factor

16559819925107279963180573885975861071762981898238616724384425798932514688349020287

of the number

$$7^{337} + 1$$

found by Ryan Propper in 2013. Setting records of this size is really only feasible with a cluster, where many thousands of curves are tried, with high ECM bounds. It may be difficult to find 70+ digit factors, and even 60+ digit factors, without a large amount of total time invested in such an environment. A more modest goal is to obtain 50+ digit factors (of numbers out of reach of GNFS, or its relation the special number field sieve, to be able to say that only ECM would likely have found the factor). This was achieved

(see section 5.4) - two 57 digit factors (of 200+ digit numbers) were found. The highest B_1 used was 400e6, with B_2 at 3300e9.

5.3 Testing the Implementation on Fionn

The implementation was set up on Fionn, the main cluster at ICHEC. Batches of various sizes were run and all program log data was recorded. Each batch included a short run to remove small factors.

5.4 Results

The largest prime factor found was the 57 digit prime factor

$$675047857067159607640250455245491501526526277140638512677$$

of the 248 digit number

$$(5^{228} + 3 \cdot 197^{110})/227812$$

using the Hessian curve

$$aX^3 + Y^3 + Z^3 = dXYZ$$

where $d = 26511$, $a = 231136413353$. The factorization of the group order of the elliptic curve over $\mathbb{F}_p$, with p equal to the above prime, is

$$2 \cdot 3^2 \cdot 58211 \cdot 73757 \cdot 824039 \cdot 13582747 \cdot 17027609 \cdot 74341063 \cdot 99190781 \cdot 6215336273.$$

The stage 1 bound used was 110,000,000 and the effective stage 2 bound was 1,151,280,345,600. Note that only 1,152 curves were run at these bounds to find the factor, far fewer than the 17,899 recommended for t55 - although this may have just been good luck (see the section on effectiveness). In addition, another (slightly smaller) 57 digit factor was found in a different run. Several 56 digit factors were found, in addition to several more 50+ digit factors smaller than this. Many smaller 40+ digit factors were found. Below is a table of the 50 largest primes found.

#	Digits	Factor	Number
1	57	675047857067159607640250455245491501526526277140638512677	$5^{228} + 3 \cdot 197^{110}$
2	57	558472654590361219396469477851602718868960566241161133359	$5^{222} + 3 \cdot 208^{110}$
3	56	25891817048987861160343065535165107858279698356935419719	$5^{182} + 3 \cdot 197^{110}$
4	56	24730861893255595347842920748047655669399212763645583607	$5^{28} + 3 \cdot 197^{110}$
5	56	16229921235478385574115452647690475125077407657479808919	$5^{196} + 3 \cdot 197^{110}$
6	55	3032735949050862506751698727169116961966449767216255779	$5^{122} + 3 \cdot 199^{110}$
7	55	1050619039713833417489844719624752724797215811371507583	$7^{56} + 3 \cdot 288^{110}$
8	54	237824884577232763884578536520487927754822702235218279	$5^{134} + 3 \cdot 208^{110}$
9	54	118866633548466488930282784455739192498657233037257383	$5^{36} + 3 \cdot 204^{110}$
10	53	56877089555815609472373181068457644471831426528864349	$5^{216} + 3 \cdot 198^{110}$
11	53	11398257998616755795224922548021024633789562392769347	$5^{232} + 3 \cdot 198^{110}$
12	52	3651313666649154688359553805753984257052900134582801	$5^{170} + 3 \cdot 208^{110}$
13	52	2435317152005744934766123177503053706183226210333771	$5^{66} + 3 \cdot 204^{110}$
14	51	703852639788595575479147681916984550331877196815199	$7^{36} + 3 \cdot 288^{110}$
15	51	406562265439225782656519894214306989722840655702413	$7^{114} + 3 \cdot 278^{110}$
16	51	296930303347206663189772469253323573457225235526431	$5^{52} + 3 \cdot 208^{110}$
17	51	105100057475669269218834436749307049581710502813033	$5^{204} + 3 \cdot 197^{110}$
18	50	91897382899334194740295370277844164475546181738119	$5^{104} + 3 \cdot 204^{110}$
19	49	8850840043069444850215419764070211989289232812903	$5^{232} + 3 \cdot 208^{110}$
20	49	4973911214568358735476535690524823464938031239539	$7^{196} + 3 \cdot 284^{110}$
21	49	4750339033313281064661859958432795423597520502249	$5^{138} + 3 \cdot 204^{110}$
22	49	2888675188950675882446838011434189450583761074331	$5^{116} + 3 \cdot 199^{110}$
23	49	1134287807665284339540358596227237401858984361899	$5^{232} + 3 \cdot 197^{110}$
24	49	1120579303117700558866126445059311944003137218979	$5^{70} + 3 \cdot 197^{110}$
25	48	931929676797391765983563583338285758765703458267	$5^{70} + 3 \cdot 197^{110}$
26	48	241519649502872521603045233730124093512423949911	$7^{98} + 3 \cdot 284^{110}$
27	48	191425859656491386789912954039146980305947642633	$5^{144} + 3 \cdot 208^{110}$
28	47	87592566228546933725435131717032313141313093353	$7^{64} + 3 \cdot 284^{110}$
29	47	67506714859780990135908157138589937519918919537	$7^{224} + 3 \cdot 288^{110}$
30	47	65079888623128724001681676043519621366011891393	$5^{26} + 3 \cdot 204^{110}$
31	47	43063706696642286554240852059521245683100234701	$5^{228} + 3 \cdot 197^{110}$
32	47	29295861349155921177664638604770286198412154523	$5^{208} + 3 \cdot 198^{110}$
33	47	19217178211776444840497044372157987296209369313	$7^{96} + 3 \cdot 288^{110}$
34	47	17920093667475575280104744534827230436078870831	$5^{188} + 3 \cdot 204^{110}$
35	46	6270662855930857199506447667018571671511359533	$7^{218} + 3 \cdot 278^{110}$
36	46	5704949175354278189410666679215214115582172111	$5^{56} + 3 \cdot 288^{110}$
37	46	5291548102290430197799015168664949626414358307	$5^{114} + 3 \cdot 199^{110}$
38	46	2833783284490531080814066153258676387349432823	$5^{132} + 3 \cdot 199^{110}$
39	46	2578340195018446589344731231912917920815427969	$7^{168} + 3 \cdot 202^{110}$
40	46	2198770815098749221063954425513077689044508631	$7^{34} + 3 \cdot 288^{110}$
41	46	2036555753917153943723306791354172854262983831	$5^{64} + 3 \cdot 204^{110}$
42	46	1842495331228826238740513860862390928088664659	$5^{26} + 3 \cdot 204^{110}$
43	46	1146113923593892302833738937717603268186616563	$5^{174} + 3 \cdot 197^{110}$
44	45	765624952011173283661844049930684310670369707	$7^{210} + 3 \cdot 282^{110}$
45	45	463362146044750707295350980899089766856058343	$5^{126} + 3 \cdot 208^{110}$
46	45	383039637998690710646436044461714836708557869	$5^{186} + 3 \cdot 204^{110}$
47	45	314543418055529178521596104132761580451751299	$5^{194} + 3 \cdot 208^{110}$
48	45	131419397263140866118229814930748266915668759	$5^{158} + 3 \cdot 204^{110}$
49	44	96630959562353588228790224310553907850343417	$5^{78} + 3 \cdot 208^{110}$
50	44	86578582354966191909696933428975608700374603	$7^{64} + 3 \cdot 284^{110}$

Table 2: Top 50 prime factors found

	Cost (milliseconds) for given B_1				
Digits	1e6	3e6	11e6	43e6	110e6
50	4790	14695	53602	208198	528966
100	7347	22246	81760	315167	807941
150	9485	28657	106112	410702	1049398
200	13338	40201	151897	579354	1467510
250	17160	51433	208026	735962	1889721
300	21824	65442	238649	934161	2392040
350	27721	114739	337054	1183745	3022484
400	32682	192005	356554	1391663	3564498

Table 3: Stage 1 Timings on 2.66GHz 64-bit desktop PC for twisted Edwards curves, using author’s program

Remark 19. As an example of the possible difference the FFT continuation makes, the largest factor found by an implementation of ECM of Brier and Clavier in [BC10] was of 46 digits. They also give a 43 digit factor and provide the factorisation of the elliptic curve group order over the corresponding prime field for the curve used. Given that the ratio of the largest prime factor to the second largest prime factor is less than 100, we might suspect that stage 2 was indeed implemented (since the largest factor would be considered large for stage 1), but not using the FFT continuation - a ratio of less than 100 is far smaller than can be achieved with the FFT continuation. There were many factors of 46 digits and larger found using the implementation of this thesis, and we can likely put this difference down to the fact that the FFT continuation was used.

5.4.1 Speed

Some timings for stage 1 using both twisted Edwards and twisted Hessian curves were obtained on a 2.66GHz desktop PC running Windows 7 x64. The Windows build of the implementation uses MPIR 2.6.0 (which is a modification of GNU-MP, which is Linux-based, to facilitate Windows). The above timings are not necessarily 100% reliable, as the operating system has the final say on whether a program is allowed to run entirely uninterrupted at the lowest level. The same tests (same numbers/bounds) were run on Fionn, where the timings are more likely to be consistent. Note that the processors/cache on Fionn are probably of much higher quality than those of the desktop pc used, even though the clock speed is lower (2.4GHz). The tests were repeated on Fionn, with the same numbers and bounds, using GMP-ECM. Both the implementation and GMP-ECM were compiled to use GMP 6.0.0a.

Remark 20. Note that GMP-ECM seems to only allow the degree of F , the stage 2 modulus polynomial, to be a power of 2. This allows the use of the ‘ $x^d + 1$ ’ trick mentioned in section 3.2 of [ZD06], which saves a factor of 2 in polynomial multiplication when using the Kronecker-Schönhage method. By oversight, this was not done in the implementation, meaning polynomial multiplication was half its potential speed. This has a knock-on

	Cost (milliseconds) for given B_1				
Digits	1e6	3e6	11e6	43e6	110e6
50	4415	13291	51215	189931	482618
100	6926	21325	77267	300644	770018
150	8938	27051	98530	386818	979713
200	12995	39124	149136	561742	1432410
250	16645	50388	182895	710613	1839109
300	21497	64896	238633	923974	2376646
350	27831	111961	306884	1191358	3057028
400	32526	193456	399501	1366344	3502925

Table 4: Stage 1 Timings on 2.66GHz 64-bit desktop PC for twisted Hessian curves, using author's program

	Cost (milliseconds) for given B_1				
Digits	1e6	3e6	11e6	43e6	110e6
50	2710	7910	29000	113510	290400
100	4240	12610	46230	180860	461850
150	5490	16460	60380	235710	602830
200	7600	22800	83620	326900	837200
250	9850	29640	108470	423850	1084710
300	12480	37560	137260	537380	1372780
350	15930	47840	175250	685370	1752690
400	18920	56750	207980	812300	2082030

Table 5: Stage 1 Timings on Fionn for twisted Edwards curves, using author's program

	Cost (milliseconds) for given B_1				
Digits	1e6	3e6	11e6	43e6	110e6
50	2530	7450	27330	106810	273540
100	4010	12040	43970	171700	439180
150	5250	15710	57400	223740	572200
200	7420	22190	81380	318230	814310
250	9480	28410	104160	406480	1040190
300	12420	37240	136520	533650	1365220
350	15970	47800	175250	684990	1751310
400	18910	56620	207970	811700	2077680

Table 6: Stage 1 Timings on Fionn for twisted Hessian curves, using author's program

	Cost(milliseconds) for given B_1				
Digits	1e6	3e6	11e6	43e6	110e6
50	896	2632	9684	39058	96942
100	1764	5364	20037	77160	201052
150	2784	8288	30493	119583	306243
200	4564	13848	50527	197932	507123
250	5892	17813	65244	255771	655116
300	7812	23373	86037	336997	863873
350	10228	30777	113115	443391	1140639
400	16269	48967	179751	705448	1806532

Table 7: Stage 1 Timings on Fionn for GMP-ECM

effect to polynomial reciprocal and scaled remainder tree, both of which use polynomial multiplication, the latter heavily. This meant that the implementation was not competitive with GMP-ECM in stage 2. So rather than give timings for different stage 2 bounds, it is perhaps more instructive to compare the program output for the implementation and GMP-ECM for a single stage 2 run, for a common input size (the output of both programs is similar in verbose mode). At the very least, the speed of stage 2 in the implementation is reasonable and provides the function that stage 2 should, and we know there is likely an easy improvement via implementing the $x^d + 1$ trick. Note that these two runs were done using the front-end node on Fionn, which may have different processors to the production nodes.

5.4.2 Effectiveness

Effectiveness is where the implementation already has an edge over GMP-ECM. Although a slight improvement, the implementation empirically finds more primes for a given bound. Since this was effectively already shown for a $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curve vs GMP-ECM in [BBLP13] (figures 9.1 and 9.2), here we show statistics for a $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curve vs a typical GMP-ECM curve (a Suyama curve with $\sigma = 4007218240$), and where a random sample of primes congruent to 1 mod 3 are chosen, so that at least torsion $\mathbb{Z}/3\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ injects into the elliptic curve group order over $\mathbb{F}_p$ for the implementation. See figures 5 and 6.

Remark 21. Interestingly, if augmented with curves #1 through #15 together with curve #28 from the list of $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves given in appendix A.4, the PARI/GP script given in appendix A.6 finds 100% of primes less than 2^{20} using only stage 1 and a bound of 512. The number of curves required to do this could probably be reduced - this set was not chosen particularly carefully.

5.4.3 Memory Usage

The memory usage for stage 1 only becomes non-negligible if either windowing is used for the scalar multiplication, or the scalar multiple itself is computed in one go by the product tree algorithm. Windowing was not used in the implementation and although the scalar multiple is computed all at once, this aspect is easy to re-work to minimize the memory cost of stage 1, at the possible expense of a small amount of extra time, so memory cost for stage 1 is not considered here.

It may be difficult to measure memory usage for stage 2 accurately but tables 8 and 9 give some rough measurements for the implementation and GMP-ECM, at least on the same machine (Fionn), using the same number of ‘stage 2 blocks’ for each similar bound. Note that the degree of F is a power of two for GMP-ECM, and this is not necessarily the case for the implementation, so the effective bounds are compared for similar, as opposed to exactly the same, stage 2 bounds. The effective bound for stage 2 in the implementation

```

GMP-ECM 6.4.4 [configured with GMP 6.0.0, --enable-asm-redc] [ECM]
Running on fionn2
Input number is 3242123362420564460491140026297872939506811720814149554381371
45780334561150691419034289968504358180300464693090283117748986750754973808401
68651837533238245630057471127121640060747208160383815917734177244620336848646
801416322696907062191705127939331 (248 digits)
Using MODMULN [mulredc:0, sqrredc:1]
Using B1=10000000, B2=117101665600, polynomial Dickson(1), sigma=1497768224
dF=32768, k=10, d=324870, d2=11, i0=20
Expected number of curves to find a factor of n digits:
35      40      45      50      55      60      65      70      75      80
114      649     4313    32762   279904  2653765  2.8e+07  3.1e+08  3.9e+09  4.9e+10
Step 1 took 75588ms
Using 29 small primes for NTT
Estimated memory usage: 157M
Initializing tables of differences for F took 0ms
Computing roots of F took 232ms
Building F from its roots took 2032ms
Computing 1/F took 1032ms
Initializing table of differences for G took 8ms
Computing roots of G took 180ms
Building G from its roots took 2261ms
Computing roots of G took 180ms
Building G from its roots took 2260ms
Computing G * H took 536ms
Reducing G * H mod F took 564ms
Computing roots of G took 180ms
Building G from its roots took 2260ms
Computing G * H took 544ms
Reducing G * H mod F took 560ms
.
.
.
Computing roots of G took 180ms
Building G from its roots took 2260ms
Computing G * H took 536ms
Reducing G * H mod F took 568ms
Computing roots of G took 180ms
Building G from its roots took 2249ms
Computing G * H took 544ms
Reducing G * H mod F took 564ms
Computing polyeval(F,G) took 4084ms
Computing product of all F(g_i) took 24ms
Step 2 took 41855ms
Expected time to find a factor of n digits:
35      40      45      50      55      60      65      70      75      80
3.73h   21.19h  5.86d   44.53d  1.04y   9.88y   102.57y  1167y   14507y  184332y

```

Figure 3: Verbose program output for 248 digit ECM run using GMP-ECM

```

Number to factor:
324212336242056446049114002629787293950681172081414955438137145780334561150691
419034289968504358180300464693090283117748986750754973808401686518375332382456
300574711271216400607472081603838159177341772446203368486468014163226969070621
91705127939331 (248 digits)
Calculating lcm...
Computation of scalar multiple took 9510ms
Factoring...

B1:10000000 B2:100000000000 B2':134108620800 d1:340170 k:10

Running curve 1 of 1 (curve id is #1)

Hessian curve parameters: a: 125 d: 123
where  $aX^3 + Y^3 + Z^3 = dXYZ$ 
Stage 1 took 118310ms

Degree of F is 39424
Computing roots of F took 1190ms
Constructing F from its roots took 5670ms
Computing precision-n reciprocal of F took 3020ms
Computing roots of G1 took 1090ms
Constructing G1 from its roots took 5630ms
Computing roots of G2 took 1090ms
Constructing G2 from its roots took 5590ms
Computing product  $H * G2$  took 1100ms
Reducing  $H = H * G2 \pmod{F}$  took 2940ms
Computing roots of G3 took 1090ms
Constructing G3 from its roots took 5590ms
Computing product  $H * G3$  took 1090ms
Reducing  $H = H * G3 \pmod{F}$  took 2940ms
.
.
.
Computing roots of G9 took 1090ms
Constructing G9 from its roots took 5590ms
Computing product  $H * G9$  took 1100ms
Reducing  $H = H * G9 \pmod{F}$  took 2920ms
Computing roots of G10 took 1090ms
Constructing G10 from its roots took 5590ms
Computing product  $H * G10$  took 1090ms
Reducing  $H = H * G10 \pmod{F}$  took 2920ms
Evaluating H at the roots of F took 27700ms

Stage 2 took 137800ms
Could not find a non-trivial factor.

```

Figure 4: Verbose program output for 248 digit ECM run using the implementation

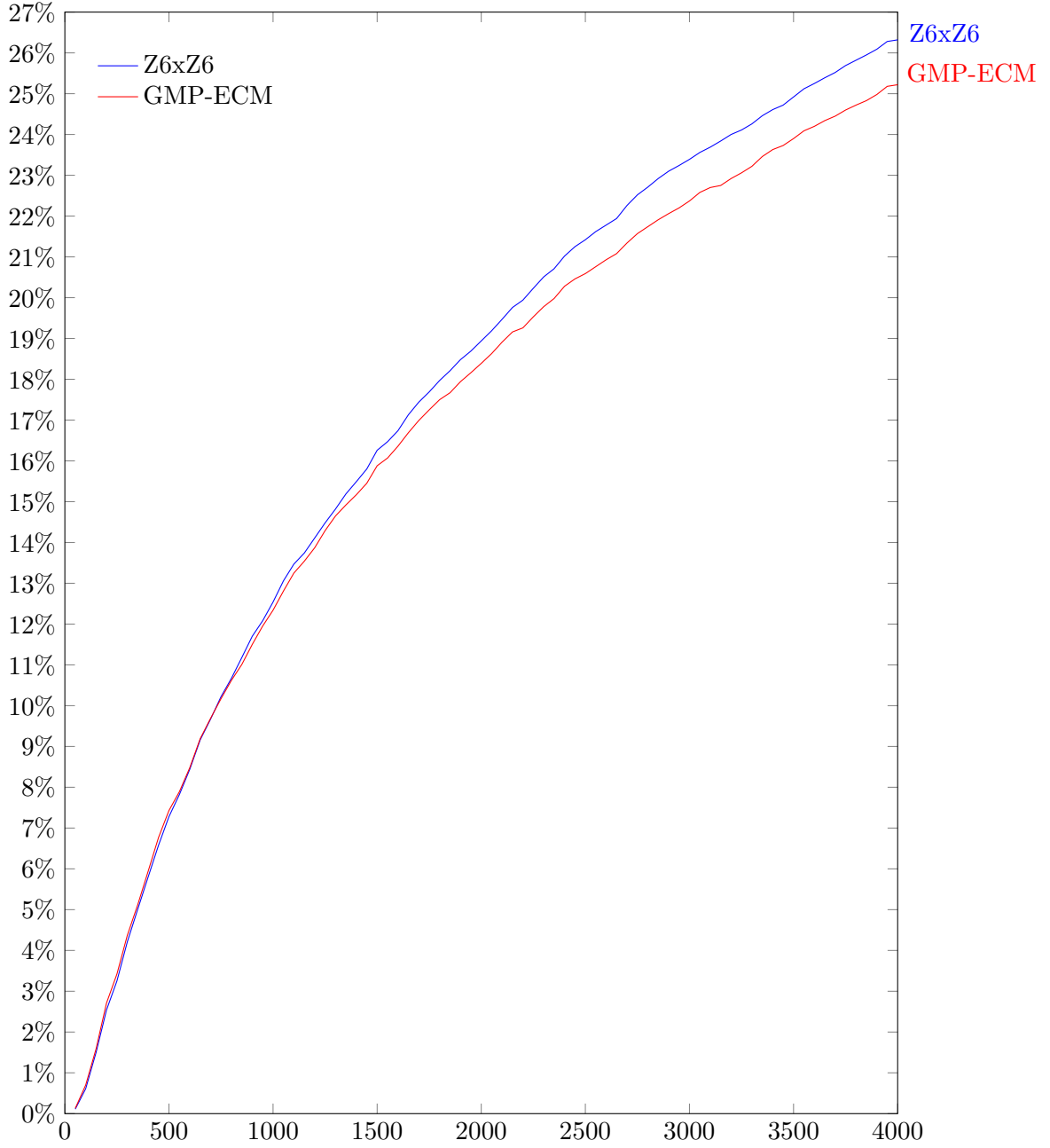


Figure 5: For a sample of 65536 30-bit primes $\equiv 1 \pmod{3}$, success probability of stage 1 of ECM for the implementation using the Jeon-Kim-Lee $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curve with $(d, a) = (123, 125)$, and GMP-ECM using the Suyama curve with $\sigma = 4007218240$. The horizontal axis is the stage 1 bound used (sampled at intervals of 50).

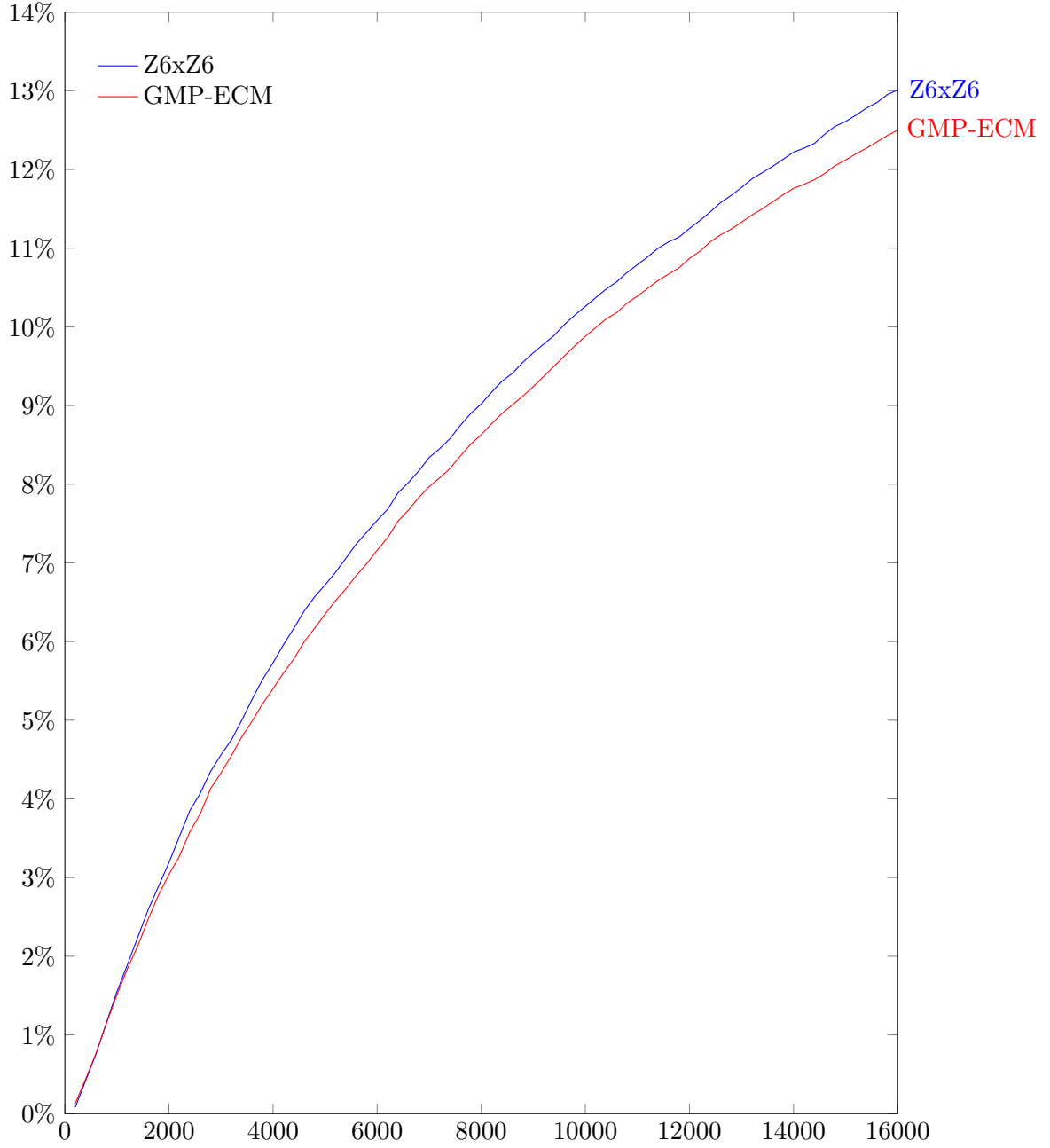


Figure 6: For a sample of 65536 40-bit primes $\equiv 1 \pmod{3}$, success probability of stage 1 of ECM for the implementation using the Jeon-Kim-Lee $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curve with $(d, a) = (123, 125)$, and GMP-ECM using the Suyama curve with $\sigma = 4007218240$. The horizontal axis is the stage 1 bound used (sampled at intervals of 200).

Target B_2	Effective B_2	#blocks	d_1	degree of F	Memory used (Mb)
1e11	108771102720	12	281010	32256	251
3e12	3064880332800	16	1385670	138240	1296
1.4e13	14164236979200	20	2661330	266112	2722

Table 8: Stage 2 memory usage on Fionn for author’s program

Target B_2	Effective B_2	#blocks	d_1	degree of F	Memory used (Mb)
1e11	140511472930	12	324870	32768	194
3e12	3104760962950	16	1345890	131072	801
1.4e13	15891877951966	20	2852850	262144	1517

Table 9: Stage 2 memory usage on Fionn for GMP-ECM

is computed using an algorithm to find the minimum d_1 composed of primes in the set $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29\}$, having a prescribed divisor of 30, for which $kd_1\phi(d_1)/2$ exceeds the requested stage 2 bound B_2 using k blocks. For one, GMP-ECM allows prime powers in d_1 , which the implementation does not (by an oversight, since allowing prime powers would allow larger B_2 for the same cost). The degree of F is what mainly determines the memory usage for stage 2.

Remark 22. The memory used by stage 2 of the implementation is slightly larger than directly predictable, since in carrying out Kronecker-Schönhage polynomial multiplication via multiplication of large integers, GNU-MP seems to internally allocate significant amounts of memory for large products, which causes the graph of memory usage to have noticeable peaks, which seem less prominent in examining the memory usage of GMP-ECM.

6 Conclusion

A new implementation of the Elliptic Curve Method of integer factorisation was written, featuring the following three main ideas:

1. The use of Twisted Edwards/twisted Hessian curves for all curve arithmetic, including stage 2.
2. The use of curves with higher torsion over quartic number fields to achieve an improvement in curve effectiveness.
3. The precomputation (using SAGE) of 700 twisted Edwards curves and over 4,800 twisted Hessian curves from the Jeon-Kim-Lee $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ and $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ families respectively for use by the implementation, where the curve parameters and point coordinates are small. Small parameters lead to faster curve arithmetic.

It was found that the implementation closely approached the speed of GMP-ECM in stage 1, with surprisingly good performance of twisted Hessian curves (for the reason given in 3.8), and was reasonably fast in stage 2 (with an easy adjustment to close the gap mentioned, see remark 20). No assembly language was implemented. If it had been, or if a faster library than GNU-MP had been used (e.g. MPFQ under certain conditions), it is very likely that speed records would have been set. As-is, although not surpassing GMP-ECM on a per-curve speed basis, it is probable that the implementation is more effective than GMP-ECM at finding primes of a certain form, a claim backed up by empirical evidence. This is relevant for attempts to factor certain families of numbers, for example Cunningham numbers, which by their structure may prescribe the existence of certain irrationalities in the prime fields of interest that allow the improvement due to quartic torsion to succeed. The implementation was tested in a high-end parallel-processing environment and the results presented. Some extra considerations/scope for further improvements are outlined below.

6.1 Generality of the Torsion Improvement

To achieve full torsion injection with the Jeon-Kim-Lee $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ and $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ families, the numbers to be factored must have a special form. If they do not have a special form, we can only expect a divisor of the group order of 16 in the first case and 18 or 6 at worst in the second, depending on the existence of suitable irrationalities in the base field. Whereas the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves at least maintain the maximum torsion possible over $\mathbb{Q}$, the special advantage of the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family breaks down for numbers of a general form. However, as noted in section 3.7, we can use the Atkin-Morain family to generate infinitely many elliptic curves with torsion $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ over $\mathbb{Q}$, together with a point of infinite order, at the expense of losing the ‘small parameter’ speedup (although regaining Montgomery arithmetic). This could be added as an option in the implementation.

6.2 Further Possible Gains

Since EECM-MPFQ's selection of $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves are a subset of the Jeon-Kim-Lee $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family and 700 of the latter were generated, an immediate improvement to EECM-MPFQ is possible - just augment the list of curves. However, EECM-MPFQ does not implement twisted Hessian arithmetic, nor does it have the advantage in stage 2 of FFT integer arithmetic used for polynomial multiplication that use of GNU-MP has over MPFQ/NTL - although adding twisted Hessian arithmetic to EECM-MPFQ would probably be easy, and it would be interesting to then compare the speed benefit of using MPFQ over GNU-MP. Also, EECM-MPFQ could be made to run stage 2 using GNU-MP, keeping MPFQ for stage 1.

The competitive performance of twisted Hessian curves, and the fact that it is possible to avoid windowing while retaining the reduced addition cost makes an ECM implementation on memory-constrained GPUs using twisted Hessian curves particularly appealing (one method of practically speeding stage 1 of ECM is to use high-performance graphics card processors for curve arithmetic, see [BCC⁺09]), even though the $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curves have lower torsion over $\mathbb{Q}$ than the $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves. However, curves in the Jeon-Kim-Lee $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/12\mathbb{Z}$ family over a quadratic number field ([JKL11b], section 3.3), with torsion $\mathbb{Z}/2\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ over $\mathbb{Q}$, may allow conversion to twisted Hessian form, allowing a higher prescribed elliptic curve group order divisor.

As already mentioned, no assembly language was written for the implementation. Due to limitations with GNU-MP, GMP-ECM features much assembly language optimisation. Since the performance of the implementation approached that of GMP-ECM (see table 6) even without assembly language, if the implementation had key routines rewritten in assembly language, there should be a significant performance improvement which should allow the implementation to surpass GMP-ECM in speed in stage 1. Alternatively, the implementation could be rewritten to use MPFQ without much work. Also, it should be possible to easily improve the implementation's stage 2 by adding the $x^d + 1$ trick, as noted in remark 20.

Projective coordinates were used throughout in the implementation, for the sake of simplicity. However, there are other coordinate systems (extended, inverted, completed, see [BBLP13], section 2) that may allow a slight performance gain. See [BL] for known curve types and formulas for curve arithmetic in various coordinate systems.

Two Jeon-Kim-Lee torsion families for elliptic curves over quartic number fields, namely $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ and $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ allow conversion to a form with fast curve arithmetic, respectively Edwards and Hessian form. There are a number of other curve types that may give an optimum representation for other families from the set - this was not investigated, as the families with highest torsion order had a natural representation.

6.3 Application in Various Scenarios

There are two common scenarios where ECM is of use - finding small to medium sized factors of numbers that are out of reach of QS or GNFS (or the special number field sieve, SNFS), and also as a cofactoring subroutine in the sieving stage of GNFS/SNFS. The implementation could be made to work for both. The application to GPU cofactoring was already mentioned. To use ECM for finding factors of large numbers, addressing the possible speedups mentioned above, it should be possible for twisted Hessian/Edwards curves to surpass the speed of the model used in GMP-ECM in stage 1 for an application such as factoring numbers from the Cunningham project. To summarise, Edwards/Hessian curves/quartic torsion can allow speed/effectiveness improvements relating to ECM for certain Cunningham number families, and speed up cofactoring in the GNFS/SNFS.

6.4 Questions that Remain Open

If it was possible to construct an elliptic curve with ‘extremely large’ torsion over any given number field, then there would be a direct implication to speed ECM. For example, even for an integer N not of any particular special form, we could construct a monic irreducible polynomial $f(x_0) = N$ for some x_0 . This then defines a number field. If we could then find an elliptic curve with large torsion over this number field, and used this curve (and perhaps others) for ECM with N , there would be a large prescribed factor in the elliptic curve group orders over prime fields $\mathbb{F}_p$ for primes p dividing N , thus reducing the amount of work to be done by ECM to obtain a smooth group order. Brier and Clavier in [BC10] give a curve having large torsion over certain cyclotomic fields, but to even find more of such curves, as well as for the case of non-cyclotomic fields, is an open question.

Edwards curves necessarily have a point of order 4 and admit faster addition formulas. It may be possible to find other curve representations with restrictions on their torsion subgroup (over $\mathbb{Q}$ or a number field) with even faster arithmetic. Edwards representation was discovered only within the last ten years or so - and the story might not end there.

A Appendix - Source Code, SAGE and Pari/GP Scripts etc

A.1 Sample C++ Function to Compute Hessian Scalar Multiple of Point, using GNU-MP Library

```
/* ScalarMultiplyHessian
 *
 * Multiply a point P = (X0:Y0:Z0) on a twisted Hessian curve by a scalar multiple S
 * d d parameter of twisted Hessian curve
 * a a parameter of twisted Hessian curve
 * P point on curve to multiply, in projective coordinates
 * SP output point [S]P
 * n we work modulo n
 * S scalar multiple
 *
 * Equation of curve:  $aX^3 + Y^3 + Z^3 = dXYZ$ 
 *
 * Doubling formula for (X1:Y1:Z1) - cost 7M + 1S + 1d
 *   B = Y1*Z1
 *   B2 = 2*B
 *   G = Y1+Z1
 *   A = G^2-B
 *   C = (A-B2)*G
 *   D = A*(Z1-Y1)
 *   E = 3*C-d*X1*B2
 *   X3 = minustwo*X1*D
 *   Y3 = (D-E)*Z1
 *   Z3 = (D+E)*Y1
 *
 * Addition formula for (X1:Y1:Z1) + (X0:Y0:Z0) - cost 12M + 1a but more like 6M
 * in practice since a,X0,Y0,Z0 are small
 *   A = X1*Z0
 *   B = Z1*Z0
 *   C = Y1*X0
 *   D = Y1*Y0
 *   E = Z1*Y0
 *   F = a*X1*X0
 *   X3 = A*B-C*D
 *   Y3 = D*E-F*A
 *   Z3 = F*C-B*E
 */
void ScalarMultiplyHessian(mpz_t d, mpz_t a, epoint* P, epoint* SP, mpz_t n, mpz_t S) {

    mpz_t A, B, C, D, E, F, G, H, Hx2;
    mpz_t B2, G2, AmB2, Z1mY1, C3, dX1B2, X1D, DmE, DaE;
    mpz_t AB, CD, DE, FA, FC, BE, mulmod, X, Y, Z;

    mpz_init(A); mpz_init(B); mpz_init(C); mpz_init(D); mpz_init(E); mpz_init(F);
    mpz_init(G); mpz_init(H); mpz_init(Hx2); mpz_init(B2); mpz_init(G2); mpz_init(AmB2);
    mpz_init(Z1mY1); mpz_init(C3); mpz_init(dX1B2); mpz_init(X1D); mpz_init(DmE);
    mpz_init(DaE); mpz_init(AB); mpz_init(CD); mpz_init(DE); mpz_init(FA); mpz_init(FC);
    mpz_init(BE); mpz_init(mulmod); mpz_init(X); mpz_init(Y); mpz_init(Z);

    mpz_mod(SP->x, P->x, n);
    mpz_mod(SP->y, P->y, n);
    mpz_mod(SP->z, P->z, n);

    // Compute exponentiation chain
    expchain* chain = new expchain();
    mpz_set(chain->S, S);
```

```

chain->L = (unsigned long long)mpz_sizeinbase(chain->S, 2); // exact for base =
power of 2

// Scalar multiplication using double & add algorithm
for (unsigned long long i = 2; i <= chain->L; i++) {
    // double  $(X1:Y1:Z1) = (SP->x:SP->y:SP->z)$ 
    mpz_mul(mulmod, SP->y, SP->z); mpz_mod(B, mulmod, n);
    mpz_mul_2exp(B2, B, 1);
    mpz_add(G, SP->y, SP->z);
    mpz_mul(mulmod, G, G); mpz_mod(G2, mulmod, n);
    mpz_sub(A, G2, B);
    mpz_sub(AmB2, A, B2);
    mpz_mul(mulmod, AmB2, G); mpz_mod(C, mulmod, n);
    mpz_sub(Z1mY1, SP->z, SP->y);
    mpz_mul(mulmod, A, Z1mY1); mpz_mod(D, mulmod, n);
    mpz_mul_ui(C3, C, 3);
    mpz_mul(dX1B2, d, SP->x);
    mpz_mul(mulmod, dX1B2, B2); mpz_mod(dX1B2, mulmod, n);
    mpz_sub(E, C3, dX1B2);
    mpz_mul_si(mulmod, SP->x, -2);
    mpz_mul(X, mulmod, D);
    mpz_sub(DmE, D, E);
    mpz_add(DaE, D, E);
    mpz_mul(Y, DmE, SP->z);
    mpz_mul(Z, DaE, SP->y);
    mpz_mod(SP->x, X, n);
    mpz_mod(SP->y, Y, n);
    mpz_mod(SP->z, Z, n);
    if (mpz_tstbit(chain->S, chain->L - i) == 1) {
        // add  $(X1:Y1:Z1) = (SP->x:SP->y:SP->z)$  and  $(X0:Y0:Z0) = (P->x:S->y:P->z)$ 
        mpz_mul(A, SP->x, P->z);
        mpz_mul(B, SP->z, P->z);
        mpz_mul(C, SP->y, P->x);
        mpz_mul(D, SP->y, P->y);
        mpz_mul(E, SP->z, P->y);
        mpz_mul(F, a, SP->x);
        mpz_mul(F, F, P->x);
        mpz_mul(AB, A, B); //mpz_mod(AB, AB, n);
        mpz_mul(CD, C, D); //mpz_mod(CD, CD, n);
        mpz_mul(DE, D, E); //mpz_mod(DE, DE, n);
        mpz_mul(FA, F, A); //mpz_mod(FA, FA, n);
        mpz_mul(FC, F, C); //mpz_mod(FC, FC, n);
        mpz_mul(BE, B, E); //mpz_mod(BE, BE, n);
        mpz_sub(X, AB, CD);
        mpz_sub(Y, DE, FA);
        mpz_sub(Z, FC, BE);
        mpz_mod(SP->x, X, n);
        mpz_mod(SP->y, Y, n);
        mpz_mod(SP->z, Z, n);
    }
}
delete chain;

mpz_clear(A); mpz_clear(B); mpz_clear(C); mpz_clear(D); mpz_clear(E); mpz_clear(F);
mpz_clear(G); mpz_clear(H); mpz_clear(Hx2); mpz_clear(B2); mpz_clear(G2);
mpz_clear(AmB2); mpz_clear(Z1mY1); mpz_clear(C3); mpz_clear(dX1B2); mpz_clear(X1D);
mpz_clear(DmE); mpz_clear(DaE); mpz_clear(AB); mpz_clear(CD); mpz_clear(DE);
mpz_clear(FA); mpz_clear(FC); mpz_clear(BE); mpz_clear(mulmod); mpz_clear(X);
mpz_clear(Y); mpz_clear(Z);
}

```

A.2 SAGE script to generate curves from $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family

```
import datetime

f = file('/media/sf_Shared/Z4xZ8_output.txt','w')
f.write("time,t,d,a,X,Y,Z,x,y,r,j\r\n")

@fork(timeout=90, verbose=True)
def getrank(E): return E.rank(proof=false)

@fork(timeout=400, verbose=True)
def getgen(E): return [E.gens(proof=false)[0],1]

num = 1
den = 1
right = true

for tk in range(10000000):
    if right:
        if num > 1:
            num = num - 1
        else:
            right = false
            den = den + 1
    else:
        if den > 1:
            den = den - 1
        else:
            right = true
            num = num + 1

    t = num/den
    if gcd(num,den) > 1 or t == 0 or t == 1 or t == -1 or num > den:
        continue

    timek = datetime.datetime.strftime(datetime.datetime.now(), '%Y-%m-%d %H:%M:%S')
    print timek, " t =",t
    v = (t^4 - 6*t^2 + 1)/(4*(t^2 + 1)^2)
    c = v^2 - 1/16

    #y^2 + x*y - c*y = x^3 - c*x^2
    E = EllipticCurve([1,-c,-c,0,0])

    r = 0
    try:
        r = getrank(E)
        if r > 0:
            T = E.torsion_subgroup()
            for k in range(T.order()):
                Pk = T.points()[k]
                if Pk.order() == 4 and Pk != [0,0] and Pk.xy()[0] != 0 and Pk.xy()[1] != 0:
                    P = Pk
                    break

            a1 = Rational(E.a1())
            a3 = Rational(E.a3())
            a2 = Rational(E.a2())
            a4 = Rational(E.a4())
            a6 = Rational(E.a6())
```

```

reset('x')
reset('y')
x,y = var('x,y')
eq = y^2 + a1*x*y + a3*y-(x^3 + a2*x^2 + a4*x + a6)

if a1 != 0 or a3 != 0:
    eq = eq.substitute(y=y-(a1*x+a3)/2).full_simplify()
    E = EllipticCurve(eq)
    P = E.point([P.xy()[0], P.xy()[1]+(a1*P.xy()[0]+a3)/2])

Px2 = P + P
if (Px2 != [0,0]):
    eq = eq.substitute(x=x+Px2.xy()[0]).full_simplify()
    E = EllipticCurve(eq)
    P = E.point([P.xy()[0]-Px2.xy()[0],P.xy()[1]])

r1 = Rational(P.xy()[0])
s1 = Rational(P.xy()[1])

da = 1 - 4*r1^3/s1^2
a = numerator(da)
d = denominator(da)

E = EllipticCurve([0,2*(a+d),0,(a-d)^2,0])
[Q,ok] = getgen(E)
if ok != 1:
    continue
reset('x')
reset('y')
x = 2*Q[0]/Q[1]
y = (Q[0]-a+d)/(Q[0]+a-d)
x = abs(x)
y = abs(y)

#####
# Verify torsion on Edwards curve over quartic number field
#R.<xx> = QQ[]
#ff = sage.schemes.elliptic_curves.heegner.make_monix(x^2-(t^4 - 6*t^2 + 1))
##print ff[0]
#K1.<a1> = NumberField([xx^2+1,ff[0]])
#K.<a2> = NumberField([K1.absolute_polynomial()]);
#E = EllipticCurve(K, [0,2*(a+d),0,(a-d)^2,0])
#
#T = E.torsion_subgroup()
#Pts = T.points()
#max = T.order()
#print "Verified torsion ",T.short_name()," over K =
#      Q(i,"sqrt(t^4-6*t^2+1),"")
#print
#####

Z=lcm(denominator(x),denominator(y))
X=x*Z
Y=y*Z
print "      ",t,",",d,",",a,",",X,",",Y,",",Z,",",x,",",y,",",r
f.write(str(timek)+","+str(t)+","+str(d)+","+str(a)+","+str(X)+","+str(Y)+","+str(Z)+","+str(x)+","+str(y)\
+", "+str(r)+","+str(E.j_invariant())+", "+str(r)\n")
f.flush()
except:
    error = true

```

```
print "Issue for t =", t,",",E.short_weierstrass_model()

f.write("\r\nFinished!\r\n")
f.write("Time taken: "+str(walltime(w)/60)+" minutes\r\n");

print "\r\nFinished!\r\n"
print "Time taken: ", walltime(w)/60, " minutes"

f.close()
```

A.3 SAGE script to generate curves from $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family

```
import datetime

f = file('/media/sf_Shared/Z6xZ6_output.txt','w')
f.write("time,t,d,a,X,Y,Z,x,y,r,j\r\n")

@fork(timeout=90, verbose=True)
def getrank(E): return E.rank(proof=false)

@fork(timeout=400, verbose=True)
def getgen(E): return [E.gens(proof=false)[0],1]

num = 1
den = 1
right = true

for tk in range(10000000):

    if right:
        if num > 1:
            num = num - 1
        else:
            right = false
            den = den + 1
    else:
        if den > 1:
            den = den - 1
        else:
            right = true
            num = num + 1

    t = num/den

    for ts in range(2):
        if ts == 1:
            t = -t

        if gcd(num,den) > 1 or t == 0 or t == 1 or t == -1/2:
            continue

        timek = datetime.datetime.strftime(datetime.datetime.now(), '%Y-%m-%d %H:%M:%S')
        print timek," t =",t
        mu = (2*t^3 + 1)/(3*(t^2))
        a4 = -27*mu*(mu^3+8)
        a6 = 54*(mu^6-20*mu^3-8)

        [d,a] = [3*numerator(mu), denominator(mu)]

        #R.<xx> = QQ[]
        #ff = sage.schemes.elliptic_curves.heegner.make_monix(x^2-(8*t^3 + 1))[0]

        #K1.<A1> = NumberField([x^2+3,ff])
        #K.<A2> = NumberField([K1.absolute_polynomial()]);
        #y^2 = x^3 - 27*mu*(mu^3 + 8)*x + 54*(mu^6 - 20*mu^3 - 8)
        E = EllipticCurve(QQ, [0,0,0,a4,a6])

        r = 0
        try:
            r = getrank(E)
```

```

if r > 0:
    #print "Positive rank for t =", t

    [Q,ok] = getgen(E)
    if ok != 1:
        continue

    [u,v] = [Q[0],Q[1]]

    x = (36*(mu^3-1)-v)/(6*(u+9*mu^2))-mu/2
    y = (v+36*(mu^3-1))/(6*(u+9*mu^2))-mu/2

    Z=lcm(denominator(x),denominator(y))
    X=x*Z
    Y=y*Z

    # Twist curve
    Y = Y*a
    Z = Z*a

    # Calculate torsion subgroup for verification
    #E2 = EllipticCurve(K, [0,0,0,a4,a6])
    #torsion = E2.torsion_subgroup().short_name()

    print "    ",t,",",d,",",a^3,",",X,",",Y,",",Z,",",x,",",y,",",r
    f.write(str(timek)+" "+str(t)+" "+str(d)+" "+str(a^3)+" "\
+str(X)+" "+str(Y)+" "+str(Z)+" "+str(x)+" "+str(y)\
+", "+str(r)+" "+str(E.j_invariant())+" "\r\n")

    f.flush()

except:
    error = true
    print "Issue for t =", t,",",E.short_weierstrass_model()

f.write("\r\nFinished!\r\n")
f.write("Time taken: "+str(walltime(w)/60)+" minutes\r\n");

print "\r\nFinished!\r\n"
print "Time taken: ", walltime(w)/60, " minutes"

f.close()

```

A.4 Sample Twisted Edwards Curves from $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ family

#	d	a	X	Y	Z
1	25921	83521	19	9537	2737
2	1681	707281	3	19642	19803
3	44182609	1766100625	11	738525	869055
4	23226064801	43617904801	11	770705	2360545
5	1003559041	6975757441	11	1354543	1610257
6	418079809	442050625	469	4394225	9925685
7	7324606372801	80246641886401	1	15213571	17497489
8	154542548161	1036488922561	38	34373405	10117405
9	3139697089	8653650625	319	79164275	82798045
10	1044727250161	1053022816561	34	49012943	49207729
11	21464689	52200625	1855	68858296	69633785
12	23804641	62742241	1529	65847273	66543609
13	656138522270401	1296864216024001	1	54427370	61150751
14	2307361	2825761	53567	51016669	93483731
15	104019601	607573201	201	100900222	101001026
16	857376961	3373402561	637	248836807	250891123
17	6493858578039361	6681375449981761	1	110211331	111962509
18	128621932321	144924114721	451	864824051	866808019
19	725779613329	732094140625	901	115120525	114611705
20	6504545260801	6569999366401	703	537109543	534168223
21	9200838241	20200652641	2891	5036475301	276804931
22	21152502721	35152125121	1729	4760760509	4765166161
23	10166959513489	12625407900625	415	1072221176	1589860649
24	174188534881	236010384481	1481	7105235799	7114843961
25	768786529	1171350625	98605	5954723004	6345478429
26	10764961	352275361	155363	3213034927	4322397313
27	26597369569	6972900390625	7719	5016025625	20988950775
28	37138300369	3082219140625	4223	19639966200	20978924280
29	5149354081	236010384481	13319	12495493289	14045697959
30	474352674970321	495066460507921	1727	55840594490	56816818135
31	138133009	244140625	4650277	61455078125	38617215625
32	52584451969	88223850625	255164	236972188475	242116052165
33	150880195489	670801950625	240916	539124025275	567164655685
34	2187793016161	17352898193761	109813	253832146009	517038627547
35	21911761	141158161	29023443	134035774763	366612082727
36	126879152401	241474942801	1051783	793950207487	885856452827
37	2052289415444401	8856579356578801	68817	745440565321	6509302928083
38	779135569	1766100625	34548565	6528322718602	951018909002
39	501446449	11156640625	19889506	168958984375	2107309831250
40	759498481	156531800881	2735617	9035513848778	9099798250102
41	13131687270289	40648594140625	130416	3200882276225	3274693742320
42	214332887521	3512479453921	1046233	2460332919187	3122881666333
43	294329035441	1998607065841	3118817	3997214131682	5837449102498
44	28235184515041	59470764169441	917311	8100999402267	10004527261567
45	6967313947489	34581750390625	663317	22298312651875	22570741308325
46	8517064396801	177294577670401	581386	16361865456011	18035201726581
47	11072116495441	115320300535441	192713	18785827792337	18888710740973
48	57121	815730721	934324639	33011806548149	42448222198811
49	2300065681	2385443281	17396243	28792762632894	28793211691413
50	18785298308401	39125037510001	1434481	17296134860169	18608317728451

A.5 Sample Twisted Hessian Curves from $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ family

#	d	a	X	Y	Z
1	123	125	10	65	5
2	345	343	4	259	49
3	129	4096	5	128	32
4	1497	3375	45	345	15
5	381	3375	56	1005	75
6	165	19683	15	513	162
7	327	216000	26	1620	600
8	-381	110592	-31	-384	672
9	1299	1259712	13	8748	6048
10	10173	5832000	118	9720	900
11	3999	35937	437	13431	231
12	687	1259712	43	13716	7776
13	501	125	-310	-285	6695
14	59043	531441	-477	243	8343
15	6063	10077696	48	2808	27864
16	4911	4913	2501	289	40834
17	8745	19683	-3720	81	91719
18	1923	8000000	22	20000	21800
19	2235	1259712	390	4104	25596
20	1803	80621568	81	17280	31968
21	107859	62099136	143	792	102168
22	537	2460375	323	83430	74925
23	2001	1000000	223	91000	19000
24	27831	16003008	322	6804	242424
25	2181	19683	7366	102627	5427
26	3945	2299968	791	42636	360096
27	6753	43243551	917	142857	41067
28	-705	85766121	-61	-11907	21609
29	4377	147197952	242	38016	165264
30	50403	791453125	185	37925	587375
31	46713	307546875	102	16875	281475
32	10137	307546875	81	286875	105975
33	-669	1331000000	-25	-24200	28600
34	40395	51395862232	65	118976	531674
35	45159	195920474112	56	220704	702768
36	1771689	7256313856	785	3872	1874048
37	1329	1331	-24455	16511	36179
38	-687	1793613375	-129	44955	164025
39	6051	4227952113	112	108339	124509
40	2193	19683	13483	172881	10449
41	130947	512000000	200	86400	1503200
42	10173	34869635163	121	460647	372438
43	20739	132651000000	106	484500	867000
44	274623	274625	4690	10985	3760445
45	150663	63808191936	-166	35964	295704
46	1581	20796875	1691	347875	875875
47	-10365	80621568	-138	473040	729216
48	13989	2072671875	889	867000	3181125
49	3075	7077888	-2303	-37248	632832
50	47337	21554582687	847	320045	2874839

A.6 PARI/GP script to factor using stage 1 of ECM and $\mathbb{Z}/4\mathbb{Z} \oplus \mathbb{Z}/8\mathbb{Z}$ curves

```
curvedata=[\
25921,83521,19,9537,2737;\
1681,707281,3,19642,19803;\
44182609,1766100625,11,738525,869055;\
23226064801,43617904801,11,770705,2360545;\
1003559041,6975757441,11,1354543,1610257];
\\ Note: More curves can be added to the above list!

eecmZ4xZ8(n, {B1}=1000, {startcurve}=1, {maxcurves}=length(curvedata[,1]) )={
local(S,tree,treepos,L,factored,i,g,d,a,X0,Y0,Z0,X,Y,Z);
[S,L] = GetChain(B1);
factored = 0; i = 1;
while(!factored && i <= min(maxcurves, length(curvedata[,1])),
[d,a,X0,Y0,Z0] = curvedata[i+startcurve-1,];
[X,Y,Z] = ScalarMultiplyEdwards(d,a,X0,Y0,Z0,n,S,L);
g = gcd(n, lift(X));
if(g > 1 && g < n, print("\nFactored in stage 1 using curve #",i); factored = 1;
);
if(!factored, print1(i,","); i += 1;);
);
if(factored, return(g);,
return("Could not find a non-trivial factor."); \\ otherwise
);
}

ScalarMultiplyEdwards(d,a,X0,Y0,Z0,n,S,L)={
local(i,X,Y,Z,A,B,C,D,E,F,G,H,J);
[X,Y,Z] = Mod([X0,Y0,Z0],n);
for(i = 2,L,
\\ double
B=(X+Y)^2;C=X*X;D=Y*Y;E=a*C;F=E+D;H=Z*Z;J=F-2*H;
[X,Y,Z] = [(B-C-D)*J,F*(E-D),F*J];
if(bittest(S,L-i)==1,
\\ add
A=Z0*Z;B=A*A;C=X0*X;D=Y0*Y;E=d*C*D;F=B-E;G=B+E;
[X,Y,Z]=[A*F*((X0+Y0)*(X+Y)-C-D),A*G*(D-a*C),F*G];
);
);
return([X,Y,Z]);
}

GetChain(B)={ \\ using product tree algorithm
local(j,tree,treepos,S,bits,L);
tree = vector(B); treepos = B;
for(j = 1,B, tree[j]=j; );
while(treepos > 1,
forstep(j = 0, treepos-1,2,
if(j + 1 < treepos,
tree[j/2+1] = lcm(tree[j+1],tree[j+2]);,
tree[j/2+1] = tree[j+1]; \\ otherwise
);
);
for(j = floor((treepos+1)/2),treepos-1, tree[j+1] = 1; );
treepos = floor((treepos+1)/2);
);
S = tree[1]; L = ceil(log(S)/log(2)); return([S,L]);
}
```

A.7 PARI/GP script to factor using stage 1 of ECM and $\mathbb{Z}/6\mathbb{Z} \oplus \mathbb{Z}/6\mathbb{Z}$ curves

```
curvedata=[\
123,125,10,65,5;\
345,343,4,259,49;\
129,4096,5,128,32;\
1497,3375,45,345,15;\
381,3375,56,1005,75];
\\ Note: More curves can be added to the above list!

ecmZ6xZ6(n, {B1}=1000, {startcurve}=1, {maxcurves}=length(curvedata[,1]))={
local(S,tree,treepos,L,factored,i,g,d,a,X0,Y0,Z0,X,Y,Z);
[S,L] = GetChain(B1);
factored = 0; i = 1;
while(!factored && i <= min(maxcurves, length(curvedata[,1])),
[d,a,X0,Y0,Z0] = curvedata[i+startcurve-1,];
[X,Y,Z] = ScalarMultiplyHessian(d,a,X0,Y0,Z0,n,S,L);
g = gcd(n, lift(X));
if(g > 1 && g < n, print("\nFactored in stage 1 using curve #",i); factored = 1;
);
if(!factored, print1(i,","); i += 1;);
);
if(factored, return(g);,
return("Could not find a non-trivial factor."); \\ otherwise
);
}

ScalarMultiplyHessian(d,a,X0,Y0,Z0,n,S,L)={
local(i,A,B,C,D,E,F,G,B2);
[X,Y,Z] = Mod([X0,Y0,Z0],n);
for(i = 2,L,
\\ double
B=Y*Z;B2=2*B;G=Y+Z;A=G^2-B;C=(A-B2)*G;D=A*(Z-Y);E=3*C-d*X*B2;
[X,Y,Z] = [-2*X*D, (D-E)*Z, (D+E)*Y];
if(bittest(S,L-i)==1,
\\ add
A=X*Z0;B=Z*Z0;C=Y*X0;D=Y*Y0;E=Z*Y0;F=a*X*X0;
[X,Y,Z] = [A*B-C*D,D*E-F*A,F*C-B*E];
);
);
return([X,Y,Z]);
}

GetChain(B)={ \\ using product tree algorithm
local(j,tree,treepos,S,bits,L);
tree = vector(B); treepos = B;
for(j = 1,B, tree[j]=j; );
while(treepos > 1,
forstep(j = 0, treepos-1,2,
if(j + 1 < treepos,
tree[j/2+1] = lcm(tree[j+1],tree[j+2]);,
tree[j/2+1] = tree[j+1]; \\ otherwise
);
);
for(j = floor((treepos+1)/2),treepos-1, tree[j+1] = 1; );
treepos = floor((treepos+1)/2);
);
S = tree[1]; L = ceil(log(S)/log(2)); return([S,L]);
}
```

Bibliography

- [BBJ⁺08] Daniel J. Bernstein, Peter Birkner, Marc Joye, Tanja Lange, and Christiane Peters. Twisted Edwards curves. In *Progress in cryptology—AFRICACRYPT 2008*, volume 5023 of *Lecture Notes in Comput. Sci.*, pages 389–405. Springer, Berlin, 2008.
- [BBLP13] Daniel J. Bernstein, Peter Birkner, Tanja Lange, and Christiane Peters. ECM using Edwards curves. *Math. Comp.*, 82(282):1139–1179, 2013.
- [BC10] Éric Brier and Christophe Clavier. New families of ECM curves for Cunningham numbers. In *Algorithmic number theory*, volume 6197 of *Lecture Notes in Comput. Sci.*, pages 96–109. Springer, Berlin, 2010.
- [BCC⁺09] Daniel J. Bernstein, Tien-Ren Chen, Chen-Mou Cheng, Tanja Lange, and Bo-Yin Yang. ECM on graphics cards. In *Advances in cryptology—EUROCRYPT 2009*, volume 5479 of *Lecture Notes in Comput. Sci.*, pages 483–501. Springer, Berlin, 2009.
- [Ber] Daniel J. Bernstein. Scaled remainder trees. URL: <http://cr.yp.to/papers.html#scaledmod>. Note: draft.
- [Ber04] D. J. Bernstein. Removing redundancy in high-precision Newton iteration, 2004.
- [Ber08] Daniel J. Bernstein. Fast multiplication and its applications. In *Algorithmic number theory: lattices, number fields, curves and cryptography*, volume 44 of *Math. Sci. Res. Inst. Publ.*, pages 325–384. Cambridge Univ. Press, Cambridge, 2008.
- [BL] Daniel J. Bernstein and Tanja Lange. Explicit-Formulas Database. URL: <https://www.hyperelliptic.org/EFD/>.
- [BL07] Daniel J. Bernstein and Tanja Lange. Faster addition and doubling on elliptic curves. In *Advances in cryptology—ASIACRYPT 2007*, volume 4833 of *Lecture Notes in Comput. Sci.*, pages 29–50. Springer, Berlin, 2007.
- [BL08] Daniel J. Bernstein and Tanja Lange. Analysis and optimization of elliptic-curve single-scalar multiplication. In *Finite fields and applications*, volume 461 of *Contemp. Math.*, pages 1–19. Amer. Math. Soc., Providence, RI, 2008.
- [CCRS14] Pete L. Clark, Patrick Corn, Alex Rice, and James Stankewicz. Computation on elliptic curves with complex multiplication. *LMS J. Comput. Math.*, 17(1):509–535, 2014.
- [DN12] Andrej Dujella and Filip Najman. Elliptic curves with large torsion and positive rank over number fields of small degree and ECM factorization. *Period. Math. Hungar.*, 65(2):193–203, 2012.
- [Edw07] Harold M. Edwards. A normal form for elliptic curves. *Bull. Amer. Math. Soc. (N.S.)*, 44(3):393–422 (electronic), 2007.
- [Eik04] Edward Vincent Eikenberg. *Rational points on some families of elliptic curves*. ProQuest LLC, Ann Arbor, MI, 2004. Thesis (Ph.D.)—University of Maryland, College Park.

- [FJ10] Reza R. Farashahi and Marc Joye. Efficient arithmetic on Hessian curves. In *Public key cryptography—PKC 2010*, volume 6056 of *Lecture Notes in Comput. Sci.*, pages 243–260. Springer, Berlin, 2010.
- [FMW12] Reza Rezaeian Farashahi, Dustin Moody, and Hongfeng Wu. Isomorphism classes of edwards curves over finite fields. *Finite Fields and Their Applications*, 18(3):597 – 612, 2012.
- [Für09] Martin Fürer. Faster integer multiplication. *SIAM J. Comput.*, 39(3):979–1005, 2009.
- [Hiş10] Hüseyin Hişil. *Elliptic curves, group law, and efficient computation*. PhD thesis, Queensland University of Technology, 2010.
- [HWCD11] Hüseyin Hişil, Kenneth Koon-Ho Wong, Gary Carter, and Ed Dawson. An exploration of affine group laws for elliptic curves. *J. Math. Cryptol.*, 5(1):1–50, 2011.
- [JKL11a] Daeyeol Jeon, Chang Heon Kim, and Yoonjin Lee. Families of elliptic curves over cubic number fields with prescribed torsion subgroups. *Math. Comp.*, 80(273):579–591, 2011.
- [JKL11b] Daeyeol Jeon, Chang Heon Kim, and Yoonjin Lee. Families of elliptic curves over quartic number fields with prescribed torsion subgroups. *Math. Comp.*, 80(276):2395–2410, 2011.
- [Kam92] S. Kamienny. Torsion points on elliptic curves and q -coefficients of modular forms. *Invent. Math.*, 109(2):221–229, 1992.
- [KM88] M. A. Kenku and F. Momose. Torsion points on elliptic curves defined over quadratic fields. *Nagoya Math. J.*, 109:125–149, 1988.
- [Kub76] Daniel Sion Kubert. Universal bounds on the torsion of elliptic curves. *Proc. London Math. Soc. (3)*, 33(2):193–237, 1976.
- [Len87] H. W. Lenstra, Jr. Factoring integers with elliptic curves. *Ann. of Math. (2)*, 126(3):649–673, 1987.
- [LL93] A. K. Lenstra and H. W. Lenstra, Jr., editors. *The development of the number field sieve*, volume 1554 of *Lecture Notes in Mathematics*. Springer-Verlag, Berlin, 1993.
- [Maz77] B. Mazur. Modular curves and the Eisenstein ideal. *Inst. Hautes Études Sci. Publ. Math.*, (47):33–186 (1978), 1977.
- [Mon92] Peter Lawrence Montgomery. *An FFT extension of the elliptic curve method of factorization*. ProQuest LLC, Ann Arbor, MI, 1992. Thesis (Ph.D.)—University of California, Los Angeles.
- [Nic07] W. Keith Nicholson. *Introduction to abstract algebra*. Wiley-Interscience [John Wiley & Sons], Hoboken, NJ, third edition, 2007.
- [Pol74] J. M. Pollard. Theorems on factorization and primality testing. *Proc. Cambridge Philos. Soc.*, 76:521–528, 1974.
- [Pom85] Carl Pomerance. The quadratic sieve factoring algorithm. In *Advances in cryptology (Paris, 1984)*, volume 209 of *Lecture Notes in Comput. Sci.*, pages 169–182. Springer, Berlin, 1985.

- [Pom08] Carl Pomerance. Smooth numbers and the quadratic sieve. In *Algorithmic number theory: lattices, number fields, curves and cryptography*, volume 44 of *Math. Sci. Res. Inst. Publ.*, pages 69–81. Cambridge Univ. Press, Cambridge, 2008.
- [Rab10] F. Patrick Rabarison. Structure de torsion des courbes elliptiques sur les corps quadratiques. *Acta Arith.*, 144(1):17–52, 2010.
- [Sil09] Joseph H. Silverman. *The arithmetic of elliptic curves*, volume 106 of *Graduate Texts in Mathematics*. Springer, Dordrecht, second edition, 2009.
- [SS71] A. Schönhage and V. Strassen. Schnelle Multiplikation grosser Zahlen. *Computing (Arch. Elektron. Rechnen)*, 7:281–292, 1971.
- [Sti95] John Stillwell. Elliptic curves. *The American Mathematical Monthly*, 102(9):pp. 831–837, 1995.
- [Sut] Andrew V. Sutherland. Torsion subgroups of elliptic curves over number fields. URL: <https://math.mit.edu/~drew/MazursTheoremSubsequentResults.pdf>.
- [Tat66] John Tate. Endomorphisms of abelian varieties over finite fields. *Invent. Math.*, 2:134–144, 1966.
- [Vél71] Jacques Vélú. Isogénies entre courbes elliptiques. *C. R. Acad. Sci. Paris Sér. A-B*, 273:A238–A241, 1971.
- [Was08] Lawrence C. Washington. *Elliptic curves, Number theory and cryptography*. Discrete Mathematics and its Applications (Boca Raton). Chapman & Hall/CRC, Boca Raton, FL, second edition, 2008.
- [Wil95] Andrew Wiles. Modular forms, elliptic curves, and Fermat’s last theorem. In *Proceedings of the International Congress of Mathematicians, Vol. 1, 2 (Zürich, 1994)*, pages 243–245. Birkhäuser, Basel, 1995.
- [ZD06] Paul Zimmermann and Bruce Dodson. 20 years of ECM. In *Algorithmic number theory*, volume 4076 of *Lecture Notes in Comput. Sci.*, pages 525–542. Springer, Berlin, 2006.